

Zabu MiningLog 2.0	2
0. MiningLog 2.0 - Overview	4
1. Roadmap	8
2. Repo & tech stack	13
3. Data Sources – Game UI & Logs	16
4. Backend – Python Agent (Architecture & Responsibilities)	26
5. Frontend – React UI	36
6. Electron Main – Host Process	41
7. System Flow & Domain Events	48
8. API & DTO Contracts (REST + WebSocket + SSE)	57
Contracts v1	67
9. Data Model - SQLite	74
9.8. Run Cost Segments	89
10. Tech decisions & tradeoffs	94
11. Future – Cloud backend (Spring) & sync	100

Zabu MiningLog 2.0

Opis

Opisz cel tej przestrzeni w maksymalnie dwóch zdaniach.

Wyszukaj tą przestrzeń

Filtruj wg etykiety

i Są to wszystkie etykiety używane w tej przestrzeni. Wybierz etykietę, aby wyświetlić listę zawartości, do której została zastosowana etykieta.

Lista etykiet

Wraz z etykietowaniem treści przez siebie i twój zespół, ten obszar będzie wypełniał się i wyświetlał najnowsze aktualizacje.



Ostatnio aktualizowana zawartość

i Poniższa lista będzie automatycznie aktualizowana za każdym razem, gdy ktoś w Twojej przestrzeni utworzy lub zaktualizuje zawartość.

- Contracts v1**
sty 11, 2026 • nadesłane przez [Patryk Cieszyński](#)
- 4. Backend – Python Agent (Architecture & Responsibilities)**
sty 09, 2026 • nadesłane przez [Patryk Cieszyński](#)
- 7. System Flow & Domain Events**
sty 09, 2026 • nadesłane przez [Patryk Cieszyński](#)
- 8. API & DTO Contracts (REST + WebSocket + SSE)**
sty 09, 2026 • nadesłane przez [Patryk Cieszyński](#)
- 9.8. Run Cost Segments**
sty 09, 2026 • nadesłane przez [Patryk Cieszyński](#)
- 9. Data Model - SQLite**
sty 09, 2026 • nadesłane przez [Patryk Cieszyński](#)

6. Electron Main – Host Process

sty 09, 2026 • nadesłane przez [Patryk Cieszyński](#)

1. Roadmap

sty 09, 2026 • nadesłane przez [Patryk Cieszyński](#)

0. MiningLog 2.0 - Overview

sty 09, 2026 • nadesłane przez [Patryk Cieszyński](#)

11. Future – Cloud backend (Spring) & sync

sty 09, 2026 • nadesłane przez [Patryk Cieszyński](#)

0. MiningLog 2.0 - Overview

0. MiningLog 2.0 – Overview

Goal

MiningLog 2.0 is a local tracking tool for Entropia Universe that:

- Continuously collects in-game data in the background:
 - player position from the compass (OCR),
 - claim information from deed windows (OCR) and chat.log,
 - finder usage / probe drops (finder UI state + OCR where needed),
 - run-related events (drops, claims found, extraction, loot).
- Stores structured data in a local SQLite database.
- Exposes a modern desktop UI (map/overlays, history, analytics).
- Runs **offline by default**.

In the future, the local client may optionally synchronize anonymized data to a separate cloud backend (Spring + online DB) for backup and aggregated stats. This is a **separate project** and a late feature, not a blocker for the local 2.0 release.

High-level Architecture

MiningLog 2.0 runs as **three processes** and presents itself as one coherent desktop app with multiple windows.

Python Agent (local backend)

Responsibilities

- Captures calibrated screen regions from the Entropia client:
 - compass ROI (Lon/Lat),
 - finder panel ROI (probe/ammo per drop, status, enabled modes),
 - deed window ROI (claim details; can be user-toggleable).
- Performs OCR (Tesseract initially; optional ML OCR later).
- Parses chat.log (system messages and whitelisted patterns).
- Implements domain logic:
 - run segmentation (RunStart/Stop),
 - claim lifecycle & correlation,
 - cost estimation (TT vs paid with MU for **tool/probe costs**),

- validation and dedup.
- Persists structured data to SQLite.
- Exposes a **private local API** consumed only by Electron Main:
 - REST for queries and commands,
 - WebSocket for high-frequency positions,
 - SSE for all other domain events (resume via Last-Event-ID, DB-backed outbox).

Networking

- Binds to `127.0.0.1` on a configurable port (default e.g. `17877`).
 - The API is **not** consumed by renderer windows directly.
-

Electron Main (Host Process + single backend client)

Responsibilities

- Starts the Python Agent as a child process and owns its lifecycle (startup readiness + clean shutdown).
 - Owns the only connection to the Agent API (REST/WS/SSE).
 - Maintains a local AppState store (Zustand vanilla) that drives all windows.
 - Applies throttling, coalescing, and shaping:
 - keeps only the last 30–60 minutes of high-frequency objects in RAM (configurable),
 - batches probe drop coverage updates and event bursts before broadcasting to UI.
 - Broadcasts state updates to renderer windows via IPC.
 - Implements crash/recovery behavior:
 - on startup: if there is an ACTIVE run, resume reading chat from the run's stored chat_start_offset + file_id,
 - otherwise: start from EOF until the user starts a new run.
-

React Renderer Windows (UI)

Renderer windows are UI-only and **never connect to localhost**.

Windows (initial set)

- **Main window**: dashboard, configuration, history, analytics.
- **Minimap / Map overlay window**: always-on-top, **not click-through** (interactive).
- **Cost/Return overlay window**: always-on-top, may be **click-through** (mostly numeric HUD).

Responsibilities

- Render UI based on IPC state from Electron Main.
 - Provide configuration screens for:
 - tool_profiles,
 - HUD calibration wizard.
 - Do not talk to Agent directly (no REST/WS/SSE from renderer).
-

Map Data (Offline)

Offline map tiles / planet imagery are **in scope**:

- The app can use local tile sets per planet.
 - Planet coordinate systems differ (different origin/grid); the app maintains per-planet coordinate transforms to map game Lon/Lat into tile coordinates.
 - This is local-only (no network dependency).
-

Scope of MiningLog 2.0 (Local Client)

Included (v2.0)

- Local database (SQLite) for runs, probe drops, claims, loot events.
- Automatic data collection:
 - player position (compass OCR → WS stream),
 - probe drops (finder state detection + OCR where required),
 - claims (chat.log + deed OCR),
 - extraction/loot events from chat.log.
- Cost tracking:
 - TT + “paid” (MU) **for probes/tools/decay** based on user tool profiles,
 - loot MU valuation is out of scope (export-first).
- Desktop UI (multi-window):
 - live map overlay + main app UI,
 - run list + run detail,
 - claim history with filters (planet, time range, bbox, resource),
 - cost/return overlay.
- Electron packaging (single installer / executable; no manual backend/frontend juggling).

Out of scope (v2.0)

- Cloud / multi-user backend (Spring + online DB).
- Global statistics, leaderboards, social features.
- Loot MU valuation, price feeds, automated MU return calculations (export first; enrichment later).
- Fully automatic HUD element discovery/tracking without calibration (explicitly deferred).

1. Roadmap

Roadmap — ZabuMiningLog (ZML) v2.0 (Local-first MVP)

This roadmap is milestone-based (no calendar dates). It reflects the current architecture decisions:

- 3 processes: **Python Agent** + **Electron Main (Host)** + **React Renderer windows**
 - **WS only for positions, SSE for all other domain events** backed by `event_outbox` (Last-Event-ID)
 - Renderers **never** call the Agent directly; all UI updates go through **IPC** (`zml/*`)
 - UI includes **3 windows**: Main, Map/Minimap, Cost/Return HUD
 - **Offline tile maps / planet imagery are in scope**
 - `tool_profiles` + `hud_calibration` are edited in UI; resource mapping YAML is edited externally (file + reload)
-

Milestone A — Repo foundation, dev loop, IPC contract

Goal: reproducible dev workflow and stable cross-process contracts.

Scope

- Monorepo layout: `agent/`, `desktop/main/`, `desktop/renderer/`, `docs/`
- Typed IPC channels (`zml/*`) + payload versioning (`ipc_version`)
- Shared enums/constants between Agent/Main/Renderer (bitmasks, event names)

Definition of Done

- `npm run dev` starts Electron Main + Renderer dev server; Main can spawn Agent (or attach to a running Agent)
 - Basic TS+Python lint/format + smoke tests
 - “IPC Contract v1” doc exists (channels, request/response, push updates)
-

Milestone B — Python Agent skeleton: SQLite + REST + SSE(outbox)

Goal: stable backend “system of record” before OCR complexity.

Scope

- SQLite schema + migrations (or “create if not exists” for MVP)
- `event_outbox` table as SSE backbone (Last-Event-ID = `event_outbox.id`)
- REST: `/health`, `/meta`, `/runs/current` (minimal)
- SSE: `/sse/v1/events` streams outbox in ascending id

- Minimal idempotency via `dedup_key` on outbox rows (optional unique)

Definition of Done

- Agent starts, creates DB, answers `/health` + `/meta`
 - SSE resume works via Last-Event-ID across reconnects and restarts
 - At least one real outbox event is emitted (e.g. `config_updated` on startup)
-

Milestone C — Electron Main (Host): lifecycle, reconnect, store, IPC fan-out

Goal: Main becomes the orchestrator and the only Agent client.

Scope

- Spawn Agent + readiness poll (`/health`) + clean shutdown
- SSE client with persisted `last_event_id` (small local file/config)
- Main store (Zustand vanilla) + rolling window retention (30–60 min) for high-frequency objects
- IPC fan-out: `zml/cmd:get_bootstrap_state` , `zml/push:state_patch` , `zml/push:notice`

Definition of Done

- Main starts Agent, connects SSE, resumes from last id
 - If resume is impossible (gap/prune), Main falls back to REST snapshot and resets cursor
 - Main creates 3 windows and broadcasts state slices to each renderer
-

Milestone D — UI MVP: 3 windows + baseline state + overlay options

Goal: multi-window UX works end-to-end (even with placeholder data).

Scope

- Main Window: dashboard + diagnostics (agent connected, `last_event_id`, calibration status)
- Map/Minimap Window: placeholder grid + overlay layers (no tiles yet)
- Cost/Return HUD Window: core numbers (cost/return/drops/claims)
- Window options: always-on-top, opacity, click-through (HUD)

Definition of Done

- App launches with 3 windows behaving as one application
 - Renderers do not touch localhost ports (IPC-only)
-

Milestone E — UI configuration: tool profiles + HUD calibration

Goal: unblock OCR & cost logic with real user configuration.

Scope

- Tool profiles editor (create/edit/select active)
- HUD calibration wizard: screenshot + draw ROIs + resolution validation
- Resource mapping YAML: file path + “reload” action (no in-app editor)

Definition of Done

- UI can save `tool_profiles` and `hud_calibration` via IPC→Main→REST
 - “Reload YAML” triggers `config_updated` over SSE and refreshes categorization
-

Milestone F — chat.log tailer MVP: ClaimFound + LootEvent + Run totals (no OCR yet)

Goal: deliver value even if OCR is not ready.

Scope

- ChatTailer + regex parsers for:
 - `claim_found`
 - `loot_event_created` (TT value)
 - `claim_depleted`
- Run accounting: `total_return_tt_mpec` derived from loot sum
- SSE events: `claim_created/updated`, `loot_event_created`, `run_updated`

Definition of Done

- Feeding a sample chat.log updates UI: loot list, TT return, claim list
-

Milestone G — Compass OCR → WS positions + map dot

Goal: live position stream, not stored in DB by default.

Scope

- OCR pipeline for Lon/Lat (preprocess + normalization + confidence gates)
- WS `/ws/v1/positions` best-effort JSON (no replay)
- Main does throttling/smoothing before broadcasting to renderers

Definition of Done

- Map window shows a smoothly moving position marker without IPC overload
-

Milestone H — Finder “cheap tracking” + ProbeDrop + costs using modes_bitmask

Goal: detect drops reliably without OCR-every-frame.

Scope

- Finder cheap detection:
 - icon state (enabled/disabled resource modes)
 - scan pulse/animation cues
 - simple state machine (IDLE → SCANNING → IDLE)
- Create `ProbeDrop` on `probe_fired` using:
 - position snapshot
 - `modes_bitmask` (no “MIXED” ambiguity)
 - tool profile radius + TT/Paid cost computation
- Emit `probe_drop_created` + `run_updated` via SSE

Definition of Done

- Drops are detected without OCR backlog
 - HUD shows increasing cost (TT/Paid), drops_count grows
 - Cost scaling follows active bits in `modes_bitmask`
-

Milestone I — Correlation + Deed scan (toggle) + Claim lifecycle

Goal: bulk dropping support: fuzzy first, precise after deed.

Scope

- ClaimCreated from chat → `PENDING_SCAN` (no coords)
- Optional approximate anchoring to recent unassigned probe drops (queue matching)
- DeedScanner (toggleable): OCR deed → authoritative coords/expiry/depth/size
- Claim categorization:
 - `category` derived from resource YAML mapping
 - unknown resources → `UNDEFINED` (user can fix YAML and reload)

Definition of Done

- Map: `PENDING_SCAN` shows fuzzy search circle; `ACTIVE` shows precise point

- Claims can be filtered by category (ORE/ENMATTER/TREASURE/UNDEFINED)
-

Milestone J — Offline tile maps + planet transforms

Goal: real map rendering (in scope), but shipped incrementally.

Scope

- Planet tile set + coordinate transforms (origin/offset/scale per planet)
- Renderer architecture:
 - base tiles layer
 - overlay layer for coverage circles/claims (Canvas/WebGL preferred for many primitives)
- Fallback for unknown planet: blank grid

Definition of Done

- At least one planet works end-to-end: lon/lat → tile coords → tiles + overlays

Risk note: this can become a scope black hole. Start with one planet + one zoom level.

Milestone K — Hardening: crash/recovery, diagnostics, exports, packaging

Goal: turn MVP into a durable tool.

Scope

- Robust Main ↔ Agent recovery:
 - restart Agent
 - SSE resume via outbox
 - REST heal when necessary
- Outbox retention policy (avoid aggressive pruning)
- Diagnostics: `/meta` + UI panel; optional debug dumps to filesystem (dev-only)
- Exports: CSV (runs/claims/loot) for external MU/Excel workflows
- Packaging: single installer/executable, no manual Python setup

Definition of Done

- Packaged app runs on a clean machine and behaves correctly
 - Restart during an active run does not lose domain events (SSE/outbox covers gaps)
 - Export produces usable data for spreadsheets
-

2. Repo & tech stack

2. Repo & Tech Stack

Monorepo Structure

```
1 zabu-mining-log/
2   agent/                                # Python Agent (OCR + log tail + SQLite +
3     src/agent/                          # OCR, parsing, domain logic, event gener
4       core/                            # screen capture + calibration regions
5       capture/                         # tesseract pipelines + preprocessors
6       ocr/                             # chat.log tail + parsers
7       logtail/                         # run/claim/loot models + services
8       domain/                         # SQLite schema + repositories
9       storage/                        # FastAPI (REST + WS positions + SSE even
10      api/                             # Pydantic DTOs (Run/Claim/Drop/Loot/...)
11      dto.py
12    tests/
13    pyproject.toml
14
15  desktop/                              # Electron app (Main + Renderers)
16    main/                               # Electron Main process (single Agent cli
17      src/
18        agent-client/                  # REST + WS + SSE clients, reconnect, hea
19        store/                        # zustand/vanilla store, selectors, TTL e
20        ipc/                          # IPC contract: commands + broadcasts
21        process/                      # spawn/monitor Agent process, logs, shut
22        package.json
23        tsconfig.json
24
25    renderer/                           # React renderer UI (no direct localhost
26      src/
27        app/                          # routes/pages
28        components/
29        features/                     # map, runs, claims, filters
30        ipc/                          # IPC client (typed)
31        state/                        # UI state (local), no backend clients
32        package.json
33        tsconfig.json
34        vite.config.ts                # (or equivalent)
35
36  docs/                                 # diagrams, .drawio, ADRs, extra notes
37  README.md
38  Makefile                             # optional dev/build helper targets
39
```

Notes:

- **Renderer never connects to Agent API.** Only Electron Main does.
- API transport split:
 - REST: queries + commands
 - **WS: positions only**
 - **SSE: all other domain events**
- Main maintains only a **rolling RAM window** for high-frequency streams (default 30–60 minutes, configurable). Long-term history comes from SQLite via Agent REST queries.

Technology Stack

Local backend (Python Agent)

- Python 3.x
- FastAPI + Uvicorn
- SQLite (recommendation: start with raw SQL + `sqlite3` or `aiosqlite` ; add SQLAlchemy later only if needed)
- Pydantic DTOs

OCR & capture

- OpenCV
- pytesseract (Tesseract wrapper)
- mss (screen capture)
- pywin32 / win32gui (window handling, client rect, positioning)
- Optional later:
 - ML OCR engine (separate module behind an interface)

Desktop app (Electron)

- Electron (TypeScript recommended)
- Electron Main:
 - Agent process lifecycle (spawn, restart, shutdown)
 - Agent API clients (REST + WS + SSE)
 - Central AppState store using **zustand/vanilla**
 - Throttling, batching, TTL eviction
 - IPC broadcast to renderers

UI Renderers

- React (TypeScript)
- IPC-only data access (typed IPC client)
- Map / visualization:
 - TBD: MapLibre / Leaflet / custom canvas overlay
 - (Decision depends on how you want to render heatmap + coverage circles efficiently)

Dev Workflow (Current Plan)

Agent

```
1 cd agent
2 python -m agent.run --port 17877 --db ./data/mininglog.db
3 # or:
```

```
4 uvicorn agent.api.app:app --reload --port 17877
5
```

Electron (dev)

Single command should start everything:

- Electron Main spawns Agent (dev mode can also attach to an already-running Agent)
- Main launches renderer dev server (Vite) or loads its URL

Example:

```
1 cd desktop
2 npm run dev
3
```

Recommended behavior:

- `npm run dev` starts Electron Main
- Main starts Agent if not running (configurable)
- Main loads renderer from `http://localhost:<vite-port>`
- Renderer connects only via IPC to Main

Production packaging

In production, the Electron app will:

- bundle the React renderer build,
- start the Python Agent as a child process,
- connect Main → Agent on `127.0.0.1:<port>`,
- expose state to renderers via IPC.

No manual backend/frontend juggling for the user.

3. Data Sources – Game UI & Logs

3. Data Sources – Game UI, Logs & HUD Calibration

This page describes what information we can reliably extract from the game (UI + logs), how it is affected by HUD position/scale, and how it maps to domain events and DTO fields (`Run` , `Claim` , `Resource` , `PlayerPosition`).

Key constraints

- We only see **screen pixels** and `chat.log` – there is no official API.
- All costs are **estimates** derived from in-game signals plus **user configuration** (tool profiles, TT constants).
- All internal cost calculations should use **PEC (int)** to avoid floating-point drift.

3.1 HUD Position, Scaling & Calibration

All relevant HUD elements in Entropia can be:

- moved around the screen,

Additionally compass can be resized/scaled.

MiningLog does **not** try to auto-detect these elements every frame. Instead, the app uses a **HUD calibration** step:

1. The user opens the game and arranges the HUD as they want to use it.
2. The app captures a screenshot of the game window.
3. In a calibration view, the user draws rectangles around:
 - the compass coordinates (`Lon:` / `Lat:` region),
 - the finder panel (probes/ammo count, resource type selection, status text),
 - optionally the tool HUD bar (tool name, remaining uses, PED card).
4. These regions (`x` , `y` , `width` , `height`) are stored in the local config, for example:

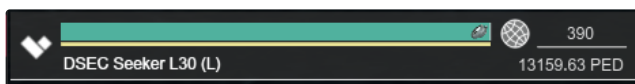
```
1 hud:
2   compass:
3     enabled: true
4     region: { x: 1600, y: 900, w: 180, h: 70 }
5
6   finder:
7     enabled: true
8     region: { x: 100, y: 500, w: 350, h: 250 }
9
10  tool_bar:
11    enabled: false
12    region: { x: 50, y: 100, w: 400, h: 60 }
13
```


As long as the HUD layout and scale do not change, OCR pipelines always use these calibrated regions.

If the player modifies HUD position/scale in the game, the calibration wizard must be run again.

For MiningLog 2.0 we assume **one HUD setup per user**; dynamic HUD tracking is explicitly out of scope for now.

3.2 Tool HUD Bar (Finder / Tool Status)



In MiningLog 2.0 this source is documented for completeness but is **not used for cost calculations**.

Example: strip showing tool name, remaining uses, and PED card value.

What we can read via OCR

- Tool name (e.g. `Rookie TerraMaster`)
- Remaining uses count (e.g. `23495`)
- Available PED on card (e.g. `13159.98 PED`)

Characteristics

- Semi-transparent background (terrain behind can vary)
- Position is user-defined but fixed after calibration
- Font is reasonably consistent but more fragile than other UIs

Possible usage (optional)

- Secondary signal to confirm which finder/extractor is equipped

Notes

- Because of transparency and background noise, this source is considered “best effort”.
 - Data from the tool bar should be treated as advisory, not authoritative.
-

3.3 Finder GUI (Finder Panel)



Finder panel is a stable, dark UI with:

- RANGE / DEPTH / TYPE
- resource icons (ore / enmatter / treasure)
- a **PROBES / AMMO** count
- status text such as:
 - Press E to initiate survey
 - Sending probe...
 - Searching...
 - You have found a resource. Follow the arrows to its location. Size: Minimal (1)

What we can read via OCR / simple processing

Primary (cheap / non-OCR):

- **ProbeFired pulse animation** on the finder radar (blue ring “impulse”).

This is used as the primary trigger for ProbeFired because it is UI-stable and avoids OCR latency/backlog.

Secondary (OCR / low-frequency):

- **PROBES / AMMO value** (units used per drop)
- **Active resource types** (which icons are lit)
- **Status text** (optional; debug / fallback only)
- **Finder hit UI fields** (when present): resource type and **Size label + index** (e.g. Minimal (1))
- Depth/range may be visible but are not considered authoritative for claim modeling (deed scan is preferred).

Characteristics

- Background and fonts are consistent and relatively OCR-friendly.
- Position is stored via HUD calibration.
- Probe/ammo count is clearly visible and is critical for cost estimation.
- **Pulse animation** occurs for a short time window (multiple frames / “several pulses”), and the exact number of pulses may vary depending on the outcome (e.g. fast hit vs normal search). Therefore, detection is **edge-based** (enter/exit hysteresis + cooldown), not “count pulses”.

Cost model input

- Probe and Universal Ammo TT values are treated as stable constants with sane defaults and optional override in local config.
- All internal cost calculations are done in **PEC/mPEC integers** to avoid floating-point drift.

Run segmentation

- **ProbeFired is triggered by the radar pulse animation** (cheap vision detector).
Status text transitions (Press E → Searching...) are not required for event generation and may be used only as a secondary validation signal.
- On ProbeFired, the system snapshots the current player position and creates a **ProbeDrop** used for coverage visualization and cost accounting.

Finder hit size hint (optional, nice-to-have)

Finder hit UI can provide **Size label + index** early. If enabled, MiningLog can compute `expires_at_estimated` using a configurable TTL-by-size table (single-mode mining only).

- This value is a **hint only**.
- Deed scan remains authoritative for:

- exact expiry (`expires_at`)
- exact coordinates (lon/lat/altitude)
- depth

Edge case: amp becomes unusable mid-run (auto mode reset)

When the last use of an amp is consumed, the game may automatically switch the finder into an “unusable” state and **disable active resource mode icons** (ore/enmatter/treasure), even though the player did not explicitly change modes.

Implications:

- MiningLog must distinguish between:
 - **FinderModeChanged (user intent)** – user toggles icons,
 - **FinderModeInvalidated (system state)** – icons drop to NONE because the tool/amp became unusable.
- A ProbeFired event may still be detected via pulse animation even if icons are not lit; mode should be taken from:
 - a. last known valid mode (if within a short grace window), otherwise
 - b. a dedicated `UNKNOWN/NONE` mode and flagged in diagnostics.

Resulting events

- **ProbeFired** – a drop attempt has started (creates ProbeDrop)
- **FinderModeChanged** – resource selection / finder mode changed by user
- **FinderModeInvalidated** – automatic mode reset due to unusable tool/amp (icons disabled)

3.4 Chat Log (`chat.log`)

Entropy writes system messages to `chat.log`. This file is not HUD-dependent and is a reliable source of discrete events.

```
[System]: You received [Energy Matter Resource Deed] x (1) Value: 0.0000 PED
[System]: You have claimed a resource! (Yellow Crystal)
[System]: You received [Yellow Crystal] x (16) Value: 0.1600 PED
[System]: This resource is depleted
```

We can parse

- **Claim hits**
 - messages indicating a resource claim was found,

- this include resource name.
- **Extractor usage**
 - messages for extraction cycles/ticks.
- **Looting**
 - messages such as:


```
You received [Yellow Crystal] x (15) Value: 0.1500 PED
```
 - provides resource name, stack size and exact TT value for that loot line.
- **Claim depletion**
 - messages indicating a claim has been fully extracted.

We do not get

- exact PED decay per probe or per extraction action,
- internal markup or hidden mechanics.

Parsing rules

Parsing is channel-gated: only `[System]` messages are considered, except a small allowlist of channel-less extraction lines (e.g. tower extraction). After passing the gate, messages must still match strict mining/extraction/loot patterns.

Intended usage

Source of truth for mining milestones:

- `ClaimFound` events → create `Claim` entities (typically `PENDING_SCAN`)
- `ExtractionTick` events → count extraction actions (optional in v2.0)
- `ResourceLooted` events → record exact resource and TT value (used for run TT return)
- `ClaimDepleted` events → mark claim finished

Validation & enrichment:

- resource name to fill `ClaimDto.resource_code` (after mapping),
- cross-check deed OCR vs chat to catch OCR errors.

Resource name mapping

Resource names from chat are mapped via a user-editable YAML (seed defaults).

The mapping can be extended automatically when new `ClaimFound(resource)` names appear (“auto-learn”), while still allowing manual overrides.

Privacy note

- Primary tables store only parsed data (resource, timestamps, TT values, etc.).

- Raw `chat.log` lines are not stored in the main analytical schema and will not be uploaded to any future cloud backend.
- Private messages are stored in `private_chat.log` ; MiningLog should never read or process that file.

3.5 Compass (Player Position)



Compass UI shows a radar plus numerical coordinates:

- `Lon: XXXXX`
- `Lat: YYYYY`

Optionally there is a planet name label (e.g. `PLANET CYRENE`).

What we read via OCR

- `Lon` and `Lat` as integers,
- optionally planet name (fallback when deed is missing or for context).

Characteristics

- Semi-transparent background, highly dependent on terrain.
- Coordinates use a stable white font; existing OCR pipeline works reasonably well.
- OCR region is defined by the calibration rectangle around the coordinate text.

Intended usage

- `PlayerPosition` events:
 - regularly sampled (throttled) and pushed over WebSocket for live UI,

- used as “current position” state.
- Compass position is used to snapshot coordinates for `ProbeFired` (coverage) and as a fallback for approximate claim location until a deed scan provides exact coordinates. Continuous movement traces are not stored in v2.0.

3.6 Resource / Claim Deed Window



The deed “OVERVIEW” window (BASIC tab) contains:

- `Depth: 19 m`
- `Size: Minimal (1)`
- `Resource: Yellow Crystal`
- `Time left: 00:59:58`
- `Position: Planet Cyrene (136167, 76554, 137)`

What we can read via OCR

- `depth` (integer meters)
- `size_label` (text) and size index in parentheses
- `resource` name
- `time_left` (remaining time as `d:hh:mm:ss`)
- `planet` name and `(lon, lat, altitude)`

Characteristics

- Dark, opaque background, OCR-friendly.
- Layout is stable across deeds (minor variation by type/planet).

- Parsing uses regexes matching known prefixes (`Depth:` , `Size:` , etc.).

Notes & limitations

Claim expiration (`expires_at`) and size classification require a **deed scan**. Without a deed scan, the claim record is incomplete and does not include expiry.

Deed windows do not provide TT value. All TT return accounting is based on parsed loot lines from `chat.log` .

Intended usage

- Primary source of structured claim data:
 - `ClaimDto.depth`
 - `ClaimDto.resource_code` (after mapping name → code)
 - `ClaimDto.size_label`
 - `ClaimDto.planet_code` , `lon` , `lat`
 - `expires_at` derived from `time_left` + scan timestamp
- Operational UX:
 - when user opens a deed window, the scanner snapshots it and updates the claim:
 - `PENDING_SCAN` → `ACTIVE`

3.7 TT Return Accounting (Run-level)

In v2.0, TT return is accounted at the **run level**, based on `ResourceLooted` lines from `chat.log` .

Claim-level TT attribution is optional/best-effort and not required, because claims may be extracted or expire without a deed scan.

3.8 What We Can / Cannot Know

We can reliably track

- Current player position (from the compass).
- `ProbeDrop` coverage points (from `ProbeFired` + position snapshot).
- Claim lifecycle events:
 - claims found (chat),
 - claim depletion (chat),
 - claim details (deed OCR when scanned).
- Resource distribution per area and planet (based on claims + deed coordinates).

- TT return for the run (sum of `ResourceLooted` TT values from chat).
- Optional: extraction action counts from chat ticks.

We estimate (with user input)

- Probe/ammo costs (from finder `PROBES/AMMO` + TT constants and tool profiles).
- Finder / amp / enhancer decay:
 - TT cost per click from tool profiles,
 - number of drops from `ProbeDrop` events.
- Extraction costs:
 - per-tick TT decay from tool profiles (if used).

All cost-related fields in `RunDto` (`total_cost_tt`, `decay_cost_tt`, `extraction_cost_tt`, etc.) are therefore **estimates**, derived from:

1. in-game UI/log signals described on this page, and
2. user-configured tool profiles and TT values.

We cannot read directly

- Raw PED decay values per action from the game.
- Any internal success/loot formulas.
- True MU values for resources.

The system is designed to be consistent and transparent, not exact to the last PEC.

4. Backend – Python Agent (Architecture & Responsibilities)

4. Backend – Python Agent (Architecture & Responsibilities)

4.1 Purpose

The Python Agent is a headless local backend responsible for collecting signals from Entropia Universe (screen + chat.log), converting them into structured domain events, persisting them to SQLite, and exposing a local API consumed by Electron Main.

Key design goal: one local backend process, one client (Electron Main), offline-first.

The Agent does not render UI and must not depend on any renderer process.

4.2 Process Topology & Ownership

MiningLog 2.0 runs as three processes:

- **Python Agent (this page)**

Screen capture + chat tail + event generation + SQLite + API.

- **Electron Main**

Starts/stops the Agent, owns app-level state, performs throttling & broadcasting, exposes IPC to renderers.

- **React Renderers (windows)**

UI only. Never connect to the Agent. They talk to Main via IPC.

Implication: the Agent API and connection limits are optimized for a single consumer (Main). We intentionally avoid “multiple renderer clients” connecting to the Agent.

4.3 Responsibilities (What the Agent MUST do)

Input acquisition

Capture calibrated screen regions from the game window (Windows):

- Compass Lon/Lat ROI (OCR)
- Finder panel ROI:
 - **cheap tracking** (icon state + scanning pulse / simple vision)
 - OCR only when needed (probe/ammo text, optional hit text)
- Deed window ROI (claim details; **optional and user-toggable**)

Tail `chat.log` incrementally (system messages and relevant public channels).

Signal processing

- OCR (Tesseract initially; optional ML OCR later).
- Parse chat patterns into domain events (ClaimFound, LootEvent, Extract ticks, etc.).
- Correlate events (ProbeDrop ↔ ClaimFound ↔ DeedScanned) using deterministic rules.
- Maintain small in-memory “latest state” caches needed for correlation:
 - last stable player position
 - last stable finder modes + probes/ammo per drop
 - pending claim queue
 - recent unassigned ProbeDrop queue

Persistence

Store canonical entities in SQLite:

- runs, probe_drops, claims, loot_events
- tool_profiles, hud_calibration, constants
- event_outbox for replayable event streaming (SSE resume)

Cost segments (run accounting overrides)

- Accept cost segment updates from Electron Main (PUT replace),
- persist them, recompute run totals deterministically,
- emit live update events (`cost_segments_updated` , then `run_updated`) via SSE.

Local API for Electron Main

- REST for querying history and current run state.
- WebSocket for high-frequency positions only (best-effort).
- SSE for all other domain events with resume (Last-Event-ID).

Reliability & correctness

- Dedup chat parsing (file identity + run start offset + rolling window where needed).
- Idempotent writes (avoid duplicates on restart).
- Deterministic money arithmetic (mPEC int + basis points MU).

4.4 Non-Responsibilities (What the Agent MUST NOT do)

- No UI rendering, no map/heatmap drawing.
- No renderer concerns (zustand stores, throttling, layout, visualization).
- No multi-client session management (single Main client is expected).
- No cloud sync in v2.0 MVP (future separate project).

4.5 Internal Architecture

The Agent is organized into five layers:

Capture Layer

- Window discovery (win32)
- MSS screen capture of calibrated ROIs
- Basic frame sanity checks (size, empty frames)

Scanner Layer

- **CompassScanner** → **player_position_updated (WS)**
- **FinderScanner** → **internal state + “probe fired” detection**
 - **Cheap tracking path (high Hz):**
 - icon enabled/disabled state (ore/enmatter/treasure)
 - scanning pulse / simple animation cues (best-effort)
 - status area “state detection” via lightweight vision (when possible)
 - **OCR path (low Hz / on-demand):**
 - probes/ammo numbers
 - optional “hit text” (Estimated size ...) as nice-to-have hint
- **DeedScanner** → **deed_scanned (optional/enable-able)**
 - Runs only when enabled and when a deed-like window is detected in the ROI

Parsing Layer

- ChatTailer + parsers (regex-based)
- Domain event builders

Domain Layer

- Event correlation rules
- Run/Claim lifecycle state machines
- Cost computation (TT + MU)

Storage & API Layer

- SQLite repositories (transactional writes)
 - event_outbox append + SSE publisher
 - FastAPI endpoints (REST + WS + SSE)
-

4.6 Concurrency Model & Performance Constraints

Reality check (important)

- OCR is CPU-heavy and inconsistent. Tesseract may block and stall your loop if run inline.
- Python has the GIL; OpenCV + Tesseract calls may still block threads.
- Screen capture + OCR + DB writes + API streaming must not run on one thread.

Recommended model

- Capture runs on a dedicated thread and schedules work.
- OCR jobs run in a bounded worker pool.
- DB writes are serialized (SQLite prefers fewer writers).
- Positions are a “fast path”: higher frequency, throttled, emitted only on change.

Suggested split

- Thread A: capture + dispatch (ROI grab, light preprocessing, queueing)
- Thread pool: OCR jobs (finder text, deed parsing)
- Thread B: chat tailer + parse
- Thread C: DB writer/outbox append (single writer queue)

Finder “cheap tracking” and snapshot semantics

Finder state is derived via a combination of:

- **cheap UI detection** (icons/visual state) at higher frequency, and
- OCR for text fields (status/probes) at a lower frequency.

To avoid race conditions around the probe fire animation and edge cases (e.g. amp becoming unusable and disabling icons automatically), ProbeDrop mode is captured using a **pre-fire snapshot**:

- The scanner maintains:
 - `modes_stable` : debounced, last-known stable `modes_bitmask`
 - a short ring buffer of `modes_candidate` with timestamps (e.g. last 0.5–1.0s)

On `probe_fired` detection at time `t0` :

- Compute `modes_bitmask_used` from the ring buffer at `t0 - Δ`, where Δ is configurable (default 100–200ms).
- If no candidate exists, fall back to `modes_stable` .
- If the mode state is unstable (recent toggles), mark `modes_quality = UNSTABLE` and prefer the previous stable value or `UNKNOWN` (safe-first).

This guarantees the ProbeDrop captures the mode that was in effect **when the action was initiated**, not what the UI shows after the animation.

Backpressure rules

- If OCR backlog grows → **drop OCR jobs / frames** (never queue infinitely).
 - Prefer fresh frames over old ones for compass/finder state.
 - **Critical nuance:** for finder drop detection, don't rely on "every-frame OCR".
Use **state-machine triggers** from cheap tracking, and OCR only to confirm / refresh slow fields.
-

4.7 Cheap Tracking (Finder) – Drop Detection Without “OCR Every Frame”

Goal

Detect `probe_fired` reliably while minimizing OCR load and avoiding false positives.

Cheap tracking signals (high Hz, low cost)

- **Mode icon state** (ore/enmatter/treasure enabled/disabled) using pixel sampling / template-ish checks.
- **Scan pulse / animation cue** around the radar circle (best-effort).
- Optional: lightweight status-area heuristics (non-OCR) if stable enough.

OCR confirmation (low Hz / on-demand)

OCR is triggered:

- on suspected `probe_fired` transition (confirm “Searching...” / scanning state),
- periodically to refresh probes/ammo per drop (e.g. every N seconds),
- optionally when “hit text” appears (size hint), but this is a nice-to-have.

State machine (pragmatic)

- Maintain `finder_state` (IDLE / SCANNING) and last transition timestamp.
- `probe_fired` is produced when:
 - cheap tracker indicates a scanning transition **AND**
 - the transition passes debouncing (e.g. stable for X ms / N consecutive frames),
 - optionally confirmed by OCR when available.
- If OCR backlog is high: still emit based on cheap tracker, but mark internally as “unconfirmed”.

Important edge case: tool becomes “unusable”

If the game auto-disables mode icons because an amp/tool hits 0 uses:

- cheap tracker sees “all modes off” abruptly while the player expected active mode.
- The Agent should treat this as a **tool invalidation** condition:

- internal flag: `finder_mode_invalidated=true`
 - optionally emit a lightweight SSE notice (e.g. `agent_notice` / `run_warning`) so UI can show “check your amp / re-enable modes”.
-

4.8 Event Outbox (SSE Backbone)

The Agent persists all non-position events into `event_outbox` :

- `id` INTEGER PRIMARY KEY AUTOINCREMENT becomes SSE Last-Event-ID
- `type, ts_ms, run_id, payload_json`

Benefits:

- SSE resume is trivial and correct.
- Debugging/replay is possible.
- Electron Main can reconnect without guessing “what happened while offline”.

Positions are not stored in outbox by default (high frequency); they are WS best-effort.

4.9 API Surface (Consumed by Electron Main)

- REST: `/api/v1/...` for state recovery and history queries
- WS: `/ws/v1/positions` only
- SSE: `/sse/v1/events` for the rest (drops/claims/loot/run updates)

Important: The Agent assumes one consumer. Rate limits and payload sizes can be tuned accordingly.

4.10 Configuration & Calibration

The Agent reads config from:

- `hud_calibration` table (active HUD profile)
- `tool_profiles` table
- `constants` table (probe/ammo TT, schema version, etc.)
- environment overrides (port, log level, dev flags)

Calibration results include:

- ROI rectangles (x,y,w,h)
- game resolution metadata (detect invalid calibration when resolution changes)

If the user changes HUD scale/position → calibration must be re-run (explicitly out-of-scope to auto-detect every frame in v2.0).

Scanner toggles:

- Deed scanner should be user-toggable (runtime change; safe default: OFF).
 - Finder OCR should be minimized; icon detection can run at higher Hz than OCR.
-

4.11 OCR Strategy (Pragmatic)

- Prefer OCR on opaque/stable panels (finder UI, deed) over transparent HUD.
- Compass Lon/Lat remains OCR-based but should be heavily normalized and throttled.
- Store “raw OCR text” only in debug logs (not in DB).

Quality gates:

- If OCR confidence is low → emit nothing for that frame (avoid poisoning state).
 - Keep parsing defensive: do not accept malformed coords.
-

4.12 Data Integrity, Idempotency, Dedup

Chat parser resume (minimal crash recovery + user-controlled backfill)

MiningLog uses a **run-scoped chat cursor** to resume parsing after a crash while keeping the user in control of whether to backfill events that occurred while the app was offline.

Run cursor fields (persisted on the `runs` row):

- `chat_file_id`: identity of the current chat.log file (implementation-defined, e.g. inode + size + mtime)
- `chat_start_offset`: byte offset captured at `RunStart` (typically EOF at start)
- `chat_last_offset` (optional): last successfully processed byte offset during this run
- `chat_last_ts_ms` (optional): timestamp of the last parsed chat line persisted for this run (diagnostics)

On RunStart

- Persist `chat_file_id` and `chat_start_offset = current EOF`.
- The parser processes only lines occurring **after** `chat_start_offset` for this run.

On Agent startup / crash recovery

- If an `ACTIVE` run exists, the system enters a *recovery decision flow*:
 - Determine `resume_from_offset = chat_last_offset ?? chat_start_offset`.

- Detect whether there is a likely “offline gap” (e.g. `now - run.updated_at_ms > threshold`, or chat file advanced significantly).
- Electron Main MUST ask the user what to do before parsing beyond the last known offset:

Recovery prompt (Electron Main)

- **Option A: Resume and sync missed events**

- Parse from `resume_from_offset` up to current EOF and persist events into the active run.
- Use run-scoped deduplication so small re-reads are safe.

- **Option B: Resume without sync (skip offline gap)**

- Set `chat_start_offset = current EOF` (or update `chat_last_offset = current EOF`) and continue normally.
- This prevents importing events that happened while the tracker was not running.

Default recommendation

- If the user is unsure, choose **Option B** (safe-first).
Importing offline history should never happen silently.

Write frequency

- `chat_last_offset` is updated **only opportunistically**, piggybacking on existing DB writes (no periodic timers, no frequent flush loops).

This model avoids complex synchronization while handling common crash scenarios and protecting the run from unintended backfilled events.

Where this logic lives

- The Agent exposes enough metadata via `/meta` or `GET /runs/current` for Main to decide whether to display the prompt.
- The user interaction itself is owned by **Electron Main** (not the Agent).

Cost segments update integrity

- `PUT /runs/{run_id}/cost-segments` **replaces all segments in one transaction** (no patch semantics in MVP).
- Validation is performed before commit (partition rules, overlap/gaps, drop_id boundaries if used).
- The write is atomic: **segments replace + recompute totals + outbox append** happen in the **same DB commit**.
 - Emit `cost_segments_updated` (SSE/outbox) and then `run_updated` (SSE/outbox) from that commit so Main/UI never observes a partial state.

4.13 Logging & Diagnostics

Minimum logging requirements:

- Agent startup: version, config profile, bind address/port
- OCR debug: optional frame dumps and preprocessed images (dev mode only)
- Parse stats: counts per event type
- Health endpoints to support Electron Main readiness checks

Recommended `/meta` endpoint exposing:

- schema_version
 - active calibration profile
 - DB path
 - agent uptime
 - active run id (if any)
 - last chat read position (offset) for the active run
-

4.14 Shutdown & Lifecycle

The Agent lifecycle is owned by Electron Main:

- Main starts the Agent as a child process.
- Main waits for `/health = ok` before opening UI.
- Main sends a shutdown signal on app exit.

Agent must:

- flush DB queues
 - close file handles (chat.log)
 - exit quickly and cleanly
-

4.15 Risks & Tradeoffs

- OCR fragility (transparent HUD, terrain variance): mitigate by calibration + preprocessing + throttling + best-effort.
 - SQLite write contention: mitigate by single-writer queue and small transactions.
 - Event correlation ambiguity (bulk dropping): accept approximate anchoring until deed scan; keep rules deterministic.
 - Scope creep (auto-detect HUD every frame, cloud sync, MU feeds): explicitly deferred.
-

4.16 Future Extensions (Non-blocking)

- Optional ML OCR plugin (ONNX runtime) for compass/finder text.
- Export pipeline (CSV/Excel) for MU computations.
- Cloud sync agent (separate service/project; different auth and privacy model).

5. Frontend – React UI

5. Frontend – React Renderers (UI Architecture, Windows & IPC)

5.1 Purpose

React Renderers are the UI layer of **ZabuMiningLog (ZML)**. They render multiple windows and display state provided by Electron Main.

Renderers **must not** connect to the Python Agent (REST/WS/SSE). They communicate **only via IPC** with Electron Main, which owns backend connectivity, state, throttling and broadcasting.

5.2 Window Types (v2.0 MVP)

ZML ships with **three** renderer windows:

1) Main Window (Dashboard / History / Config)

Responsibilities:

- Runs list + run details
- Claims history + filters
- Loot summary (TT only)
- **Tool profiles editor (UI)**
- **HUD calibration wizard (UI)**
- Diagnostics panel (Agent connection, calibration validity)

2) Map Window (Minimap)

Responsibilities:

- Live map (player + coverage + claims)
- Tile-based planet map (offline tiles)
- Basic filters/toggles (coverage on/off, claim states)

Notes:

- Not click-through (interactive)
- Always-on-top optional (user setting)

3) HUD Window (Cost / Return)

Responsibilities:

- Minimal numeric view:
 - total cost (TT / paid)

- total return (TT)
- ROI / profit (optional, TT-only)
- drops count / claims count
- Optional: small status icons (Agent connected, calibration OK)

Notes:

- **May be click-through** (intended as “numbers-only overlay”)
 - No hotkeys in MVP (optional later)
-

5.3 State Ownership Model

Electron Main owns:

- Connection to Python Agent:
 - WS positions (best-effort)
 - SSE domain events with resume (Last-Event-ID via event_outbox)
 - REST snapshot recovery
- Canonical AppState store (zustand/vanilla)
- Throttling/batching and fan-out to multiple windows
- Window registry + lifecycle (create/show/hide/always-on-top/click-through)

Renderers own:

- View/UI state (layout, selected run, filters, map zoom)
 - Derived presentation state (selectors), **not canonical domain state**
-

5.4 Data Flow Overview

1. Agent emits:

- WS positions → Main receives high-frequency updates
- SSE domain events → Main receives ordered outbox stream

2. Main:

- Applies events to AppState
- Keeps rolling RAM window (e.g., last 30–60 min for high-frequency items)
- Broadcasts to renderers via IPC (batched)

3. Renderers:

- Subscribe to IPC updates

- Render map / HUD / pages
-

5.5 IPC Contracts (Renderer ↔ Main)

5.5.1 Namespace & Versioning

- IPC channels: `zml/*`
- Protocol version in payloads: `ipc_version: 1`
- Backward-compatible changes: only additive (new optional fields) unless bump.

5.5.2 Renderer → Main (Commands)

`zml/cmd:get_bootstrap_state`

- Initial snapshot for a newly opened window.
- Returns relevant slice + metadata.

`zml/cmd:open_window`

- Opens: `main | map | hud_cost`
- Options: size preset, always-on-top, click-through (per window)

`zml/cmd:set_window_options`

- `always_on_top`, `opacity`, `click_through`, `bounds`

`zml/cmd:query_history`

- Runs list, claims list, loot aggregation (paged)

`zml/cmd:config:update_tool_profiles`

- UI-managed tool profiles

`zml/cmd:config:update_hud_calibration`

- UI-managed calibration result (ROIs + resolution metadata)

(Explicitly NOT in UI)

- Resource classification YAML is file-edited; UI may only show path + “reload”.

`zml/cmd:resources:reload_yaml`

- Main/Agent re-reads YAML and emits `config_updated` event

5.5.3 Main → Renderer (Push Updates)

`zml/push:state_patch`

- Primary update channel (batched).

- MVP: allow full slice replacement for simplicity, later switch to patches.

`zml/push:position`

- Optional: separate high-rate channel to Map window.

`zml/push:notice`

- User-facing notices:
 - backend disconnected
 - calibration invalid (resolution changed)
 - finder modes disabled unexpectedly (e.g., amp becomes unusable)
 - deed scanner disabled/enabled
-

5.6 UI-facing View Models (VMs)

Renderers should **not** bind directly to Agent DTOs. Main reshapes into UI-friendly VMs:

- `RunSummaryVm`
- `ClaimVm` (includes `resource_category: ORE|ENMATTER|TREASURE|UNDEFINED`)
- `CoverageCircleVm`
- `PositionVm`
- `TileMapVm` (planet tile metadata + transforms)

This keeps UI stable even if Agent schema evolves.

5.7 Offline Tile Maps (In Scope for v2.0)

ZML supports offline planet maps using a tile set per planet.

Tile map inputs (per planet)

- tiles directory path (local)
- tile size (px)
- zoom levels available
- coordinate transform metadata:
 - planet grid origin (lon/lat reference)
 - scale (lon/lat units → pixels or tile coords)
 - any per-planet offset rules

Rendering policy

- Map window uses tiles as base layer.

- Overlay elements (player/claims/coverage) rendered on top (Canvas/WebGL recommended).

Notes:

- Different planets may have different coordinate origins; mapping must be planet-aware.
 - If planet is unknown, map falls back to “blank grid” mode.
-

5.8 Performance Rules

- Map window must be smooth:
 - tiles: DOM is ok (img layers) but overlays should be Canvas/WebGL for many circles
 - Main throttling:
 - positions: cap broadcast to map window (e.g., 10–20 Hz)
 - SSE domain updates: batch every 100–250 ms
 - HUD cost window:
 - push updates at low frequency (e.g., 2–5 Hz) or on meaningful changes
-

5.9 Configuration UX (MVP)

In UI:

- Tool profiles editor (create/edit/select active profile)
- HUD calibration wizard (ROI capture + validate + save)

Out of UI:

- Resource YAML edited in Notepad
 - UI provides:
 - file path display
 - “Reload resources” button (sends `zml/cmd:resources:reload_yaml`)
 - basic validation error display if YAML is invalid
-

5.10 Out of Scope (v2.0 MVP)

- Hotkeys / global keyboard hooks (explicitly deferred)
- MU price feeds and automatic loot MU valuation
- Cloud sync UI
- Full replay/history for WS positions

6. Electron Main – Host Process

6. Electron Main – Host Process

This page describes the Electron **Main** process responsibilities in MiningLog 2.0 (ZabuMiningLog / `zml/*`). Main is the application host: it starts the Python Agent, owns app-level state, manages windows, and is the **only** client of the Agent API.

6.1 Purpose

Electron Main is responsible for:

- **Process orchestration:** start/stop/restart the Python Agent and wait for readiness.
- **Connectivity:** maintain REST + SSE + WS connections to the Agent.
- **AppState ownership:** keep the canonical in-memory state used by UI windows (rolling window 30–60 min).
- **IPC gateway:** expose a stable IPC contract to UI windows (`zml/*`).
- **Window management:** create, position, and configure multiple UI windows.

Key principle: **one coherent app** with multiple windows, but a **single backend client** (Main).

6.2 Process Topology (Runtime)

MiningLog runs as three processes:

1. Python Agent (backend)

OCR + chat parsing + SQLite + REST + WS + SSE.

2. Electron Main (this page)

Host process: starts Agent, consumes Agent streams, owns AppState, broadcasts to windows via IPC.

3. UI Windows (Renderer processes)

Presentation only; never connect to the Agent directly. They talk to Main via IPC.

Renderers must not know Agent URLs, ports, or auth. They only know IPC channels.

6.3 Window Model (Multiple Windows, One App)

MiningLog 2.0 uses multiple BrowserWindows:

6.3.1 Main window

- dashboard, history, analytics, configuration (tool_profiles, HUD calibration, run controls)

6.3.2 Minimap / Map window (overlay)

- always-on-top map overlay with player + claims + coverage
- **not click-through** (interactive)
- no global hotkeys in v2.0

6.3.3 HUD window (cost/return overlay)

- minimal numeric overlay (run cost/return, drops/claims counters)
- **click-through** allowed
- no global hotkeys in v2.0

Window flags (high level):

- `alwaysOnTop` : map + HUD
 - `transparent` : HUD (optional)
 - `focusable` : false for click-through HUD, true otherwise
 - `ignoreMouseEvents(true)` : HUD click-through mode
-

6.4 AppState Ownership (Main as the in-memory state owner)

Main holds the canonical in-memory state used by UI windows.

State contents (typical):

- current run summary
- recent ProbeDrops (rolling window, default 30–60 min)
- claims in current run (all, or filtered by recency)
- recent LootEvents (optional window)
- latest PlayerPosition (current only)
- configuration snapshots (tool_profiles, hud_calibration, constants)
- connection health (agent status, last SSE id, WS connected, etc.)

Memory policy:

- keep only the last 30–60 minutes of high-frequency objects
- full history always lives in SQLite (queried through REST when needed)

Store choice:

- any local store is fine as long as it's deterministic and transport-agnostic
 - do not couple state management to UI framework
-

6.5 Agent Connectivity (Main → Agent)

Main maintains three channels:

1. **REST** (`/api/v1/...`) for snapshot + history + config writes
 2. **WS positions** (`/ws/v1/positions`) for best-effort high-frequency position stream
 3. **SSE domain events** (`/sse/v1/events`) for resumable domain events via Last-Event-ID backed by `event_outbox`
-

6.7 IPC Contract (Windows ↔ Main)

All window communication goes through IPC.

Naming convention (`zml/*`):

- `zml/cmd/*` — request/response
- `zml/push/*` — Main → window broadcast
- `zml/ui/*` — window → Main user actions (optional)

Core commands (examples):

- `zml/cmd/getAppState`
- `zml/cmd/startRun` , `zml/cmd/stopRun`
- `zml/cmd/getRuns` , `zml/cmd/getRunDetails`
- `zml/cmd/updateToolProfiles`
- `zml/cmd/updateHudCalibration`
- `zml/cmd/setOverlayMode` (HUD click-through toggle)

Broadcast topics (examples):

- `zml/push/positions`
 - `zml/push/domainEventsBatch`
 - `zml/push/runSummary`
 - `zml/push/configChanged`
-

6.8 Startup & Shutdown Lifecycle (Main owns lifecycle)

6.8.1 Startup sequence

On app launch, Electron Main must:

1. **Load local config** (db path, agent port, feature flags).

2. **Spawn Python Agent** as a child process.

- Prefer passing `--port` , `--db-path` , `--log-level` .

3. **Wait for readiness:**

- Poll `GET /health` with a short interval and a hard timeout.
- If Agent exits early → surface error + offer restart.

4. **Connect streams:**

- Open SSE `/sse/v1/events` with `Last-Event-ID` if available.
- Open WS `/ws/v1/positions` (best-effort).

5. **Hydrate AppState** (cold start recovery):

- `GET /runs/current`
- if active run exists → fetch deltas (`probe-drops` , `claims` , `loot-events`)

6. **Create windows** (main + overlays), then start IPC broadcasting.

Key rule: UI windows should not show “empty state” if Main hasn’t hydrated yet. Main broadcasts a `zml/push/appReady` + initial state snapshot.

6.8.2 Shutdown sequence

On app exit, Electron Main must:

1. **Stop IPC broadcasting** (prevent late UI updates).
2. **Close streams:** terminate SSE + WS cleanly.
3. **Request Agent shutdown** (preferred: `/api/v1/shutdown` or SIGTERM).
4. **Wait for graceful exit** (short timeout), then kill if needed.
5. Ensure no orphan processes.

6.9 Throttling, Coalescing, and Backpressure (Main is the pacing layer)

Main is responsible for UI pacing. The Agent can be conservative, but Main owns:

- throttling update frequency to each window,
- batching/coalescing events,
- dropping low-value updates under pressure.

6.9.1 Positions stream handling (WS)

Positions can be high-frequency and noisy.

Policy:

- Main keeps only **latest position** per planet (no queue).

- Broadcast positions to UI at a fixed cadence (e.g. **10–20 Hz max**), regardless of WS rate.
- Optional smoothing is UI-layer, but Main should avoid spamming IPC.

6.9.2 SSE event handling (domain events)

SSE is resumable and should not be dropped in Main's internal ingestion.

But **broadcast can be coalesced**.

Policy:

- Persist last seen `event_outbox.id` in Main state.
- Ingest all events, apply them to AppState deterministically.
- Broadcast in **batches** (e.g. every 50–200ms or N events).
- If UI is slow, Main may:
 - skip per-event push and instead send an updated snapshot slice (e.g. `runSummary + recentDrops + claims`).

6.9.3 Backpressure strategy

When Main detects pressure (IPC queue backlog, renderer jank, memory growth):

- **Positions:** drop intermediate updates (keep latest only).
- **Coverage:** shrink rolling window (e.g. from 60 → 30 min) or downsample probe drops for overlay rendering.
- **SSE events:** do not drop ingestion; instead reduce broadcast frequency and send snapshots.

This keeps correctness while maintaining responsiveness.

6.10 Crash / Recovery Behavior (Host-level resilience)

6.10.1 Agent crash

If the Agent process exits unexpectedly:

1. Mark Agent status as **DOWN** in AppState.
2. Notify windows (`zm1/push/agentStatus`).
3. Attempt controlled restart:
 - spawn Agent again
 - wait for `/health`
 - reconnect SSE with stored `Last-Event-ID`
4. Heal via REST snapshot if needed.

Important: SSE resume should restore correctness even if Main missed events during downtime.

6.10.2 Main crash / app restart

On restart:

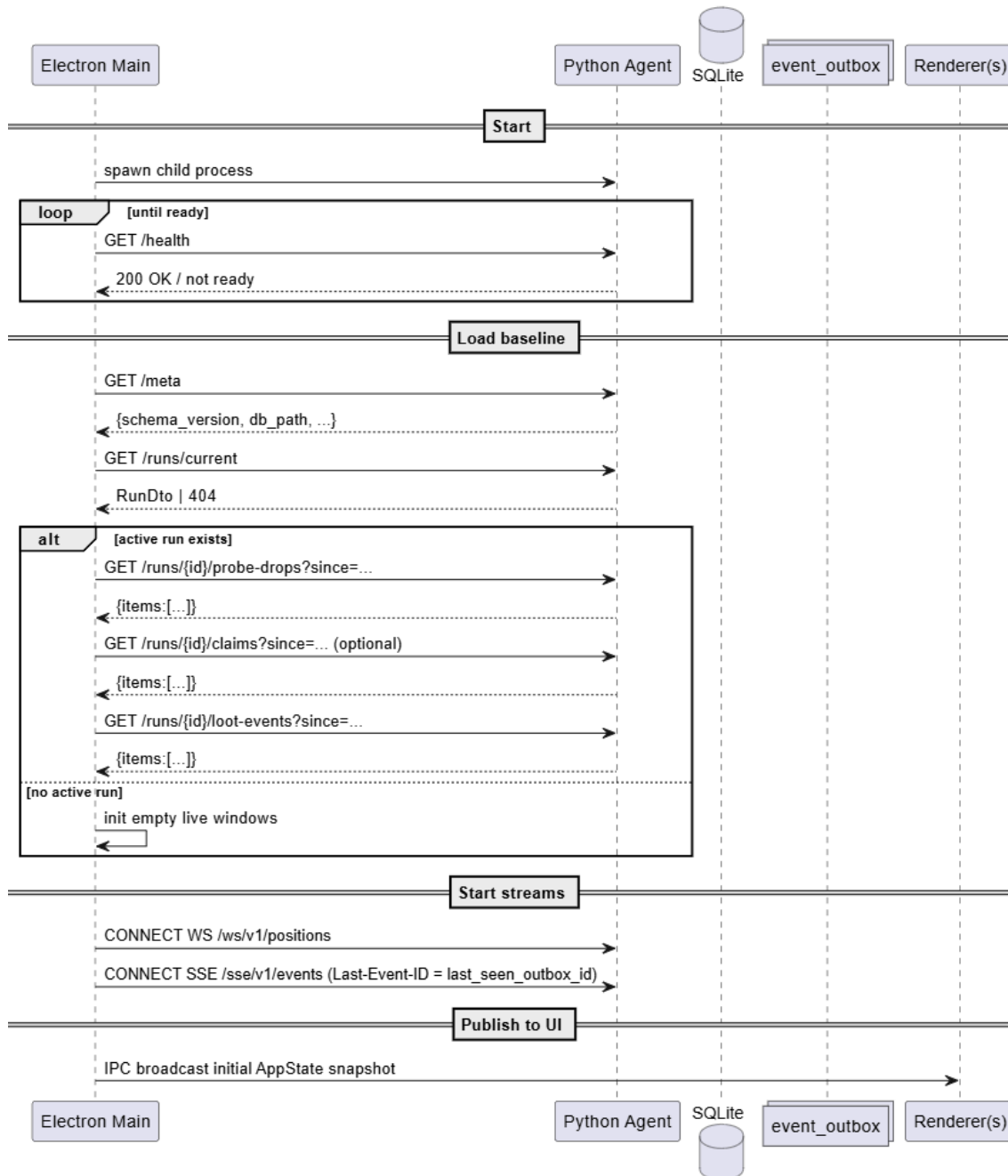
- Main loads the stored `Last-Event-ID` cursor (local file/config).
- Connects SSE with `Last-Event-ID`.
- If outbox retention is insufficient or gap detected:
 - fall back to REST snapshot + reset cursor.

6.10.3 Active run continuity

If there is an ACTIVE run in SQLite:

- Main resumes it (no “immutable runs” assumption).
- Agent parsing resumes based on `run.chat_start_offset` and file identity (as defined in DB model).

ZML Recovery: App Startup (Health -> SSE Resume -> REST Heal)



6.11 Persistence of Host Cursors (minimal, practical)

Main should persist only what it needs for reconnect:

- `last_sse_event_id` (integer)
- optional: last known agent port/config hash

This is not “analytics data”, just connectivity state.

7. System Flow & Domain Events

7. System Flow – Events, Correlation & Lifecycles

This page explains how ZabuMiningLog (ZML) turns game signals (OCR + `chat.log`) into **domain events**, persists them, and streams live updates to the desktop UI. It also defines correlation rules for bulk dropping and the lifecycle of **Run** and **Claim**.

Streaming model (important):

- **WS** is used only for **high-frequency positions** (best-effort, no replay).
- **SSE** is used for **all other domain events**, backed by `event_outbox` for resume via `Last-Event-ID`.

7.1 End-to-End Pipeline (High Level)

Inputs

Screen capture (calibrated HUD regions):

- **Compass** (Lon/Lat)
- **Finder panel**
 - **Cheap signals (high frequency):**
 - mode icon state (ore/enmatter/treasure lit/unlit)
 - **pulse / animation** detection (drop feedback)
 - **OCR signals (lower frequency):**
 - status text (idle/searching/result)
 - probes/ammo text (units used per drop)
 - *(optional / nice-to-have)* “found resource / estimated size” line
- **Deed window** (claim details; **optional / user-toggleable**)

File input:

- `chat.log` (system messages and relevant public channels)

Processing (Agent)

- Scanners capture calibrated regions and run OCR / simple vision checks.
- Chat parser tails `chat.log` incrementally and extracts mining events.
- All detected signals are converted into **internal domain events** and pushed into the Agent’s internal event bus.

- Services consume events to:
 - update in-memory state (latest position, finder state, current run),
 - persist records in SQLite,
 - publish live updates:
 - **WS** positions stream (high-frequency, best-effort),
 - **SSE** domain events (replayable via `event_outbox`).

Outputs

SQLite persistence (Agent):

- `runs`, `probe_drops`, `claims`, `loot_events`, `tool_profiles`, `hud_calibration`, `constants`, `event_outbox`

Private local API (Agent → Electron Main):

- REST for history/query and commands
- WebSocket for live positions
- SSE for all other domain events

Desktop UI delivery (Electron Main → React Renderers):

- Main holds AppState store (zustand/vanilla), throttles/batches, broadcasts via IPC
- Renderers never connect to Agent directly

7.2 Domain Events (Source → Event → Side Effects)

Event publishing model

- Player position uses **WebSocket** (positions only).
- All other events use **SSE** (`id` , `event` , `data`) backed by `event_outbox` , so Main can reconnect and resume from the last event id.
- After reconnect, Main may also “heal” state using REST queries (authoritative).

Event table

Source	Trigger	Internal domain event (Agent)	Persisted entities	Stream (Agent→Main)	Notes
Compass OCR	lon/lat changed (throttled)	<code>player_position_update</code>	none (live-only)	WS <code>/ws/v1/positions</code>	High-frequency, best-effort. No replay.

Finder (cheap)	pulse / visual “drop” signal OR stable status transition	probe_fired	probe_drops	SSE probe_drop_ created	Coverage anchor + cost input. Prefer cheap detection + state machine over constant OCR.
Finder (cheap)	mode icons changed (lit/unlit)	finder_mode_ s_changed	(optional snapshot)	(optional SSE)	Typically internal only . Used to stabilize cost logic and for later segment overrides.
Finder OCR (slow)	probes/ammo text changed	finder_unit_ s_changed	(optional snapshot)	(optional SSE)	Usually internal only . Main can show the latest via REST if needed.
Finder OCR (nice-to-have)	“found resource... estimated size ...”	finder_hit_ hint	claims (hint fields)	SSE claim_updat ed	Only reliable for single-mode mining. Hint only ; deed scan remains authoritative.
chat.log	“You have claimed a resource! ...”	claim_found	claims (PENDING_SCA N)	SSE claim_creat ed	No coordinates in chat. Claim may be anchored approximately to a recent ProbeDrop.
Deed OCR	deed window detected + parsed	deed_scanne d	update claims	SSE claim_updat ed	Authoritative for coords + expiry + size + depth.

					Overrides hints/estimates.
chat.log	"You received ... Value: ..."	resource_lo oted	loot_events + update runs totals	SSE loot_event_ created + run_updated	Loot TT is authoritative. Loot MU out of scope in v2.0.
chat.log	claim depleted message	claim_deple ted	update claims	SSE claim_updat ed	Terminal state: DEPLETED.
Scheduler	expires_at crossed (if known)	claim_expir ed	update claims	SSE claim_updat ed	Only possible if deed scan provided expiry.
System	user starts run (or auto-start on first drop)	run_started	create/update runs	SSE run_updated	Run becomes ACTIVE when started (manual) or on first detected drop (auto-start).
System	user stops run / app exit / explicit "finish"	run_finishe d	update runs	SSE run_updated	No inactivity timeout. Finish is an explicit action (or controlled by Main on shutdown).

Contract rule: Only SSE events are replayable/resumable. WS positions are best-effort only.

7.3 Correlation Rules (Bulk Dropping Friendly)

Mining workflow supports "bulk dropping" (many drops first, scanning/extraction later). Correlation focuses on deterministic, low-risk rules. When correlation is uncertain, the system must **prefer not linking** rather than linking incorrectly.

7.3.1 ProbeDrop creation (coverage anchor)

On `probe_fired`, create `ProbeDrop` with:

- `ts`
- `lon/lat` snapshot from latest compass position
- `radius_m` from selected tool profile
- `modes_mask` snapshot from the **last stable finder mode state**
- costs:
 - `cost_tt_mpec` derived from probes/ammo per drop + TT constants + tool decay
 - `cost_paid_mpec` applying MU (bps) where configured

Mode snapshot rule (anti-race)

When `probe_fired` is detected, the Agent sets `probe_drop.modes_mask` using a **pre-fire stable snapshot**, not “whatever icons did 200ms later”.

This avoids incorrect mode attribution caused by:

- UI animation timing
- automatic icon deactivation (e.g. last amp use → nodes disabled by the game)

UI handling:

- Main keeps only a rolling time window (default 30–60 min) of `ProbeDrop` in memory for rendering coverage.
- Full history is queryable from SQLite via REST.

7.3.2 ClaimFound positioning (no coordinates in chat)

`claim_found(resource)` creates `Claim(status=PENDING_SCAN)`.

To provide useful map UX before deed scan, claims may be anchored to recent probe drops.

Rule: queue matching (deterministic, low risk):

- Maintain a queue of recent **unassigned** `ProbeDrop` ids (per run).
- When `ClaimFound` occurs, assign it to the **most recent unassigned** `ProbeDrop` within a time window (e.g. 2–5 minutes).

Persist:

- `claim.drop_id`
- `claim.approx_center = drop.lon/lat`
- `claim.approx_radius_m = drop.radius_m`

Notes:

- This is an approximate “search area” marker only.
- If correlation fails, the claim remains unanchored until deed scan.

7.3.3 Finder hit hint (single-mode only, optional)

If the finder UI provides an early “Estimated size ...” line, ZML may store it as **hint fields**, without changing the claim state machine.

Recommended approach (no extra state):

- `claim.predicted_size_label`
- `claim.predicted_size_idx`
- `claim.predicted_expires_at_ms` (*derived via TTL-by-size table; single-mode only*)
- `claim.predicted_source = FINDER`

These hints are overwritten/ignored once deed scan arrives.

7.3.4 DeedScanned enrichment (authoritative)

`deed_scanned` updates an existing claim.

Matching strategy (best effort, safe-first):

- Prefer the most recent `PENDING_SCAN` claim in the same run.
- If resource name exists, require a normalized match.
- If multiple candidates exist, choose newest by `found_at`.

Update fields:

- `planet_code, lon, lat, altitude` (authoritative)
- `depth_m`
- `size_label, size_idx`
- `expires_at` (authoritative)

Deed scan always overrides estimates/hints.

7.3.5 Loot accounting (run-level)

`resource_looted` is accounted at the run level:

- `run.total_return_tt_mpec += loot.value_tt_mpec`

Claim-level return attribution is optional/best-effort and not required in v2.0.

7.4 Lifecycles (State Machines)

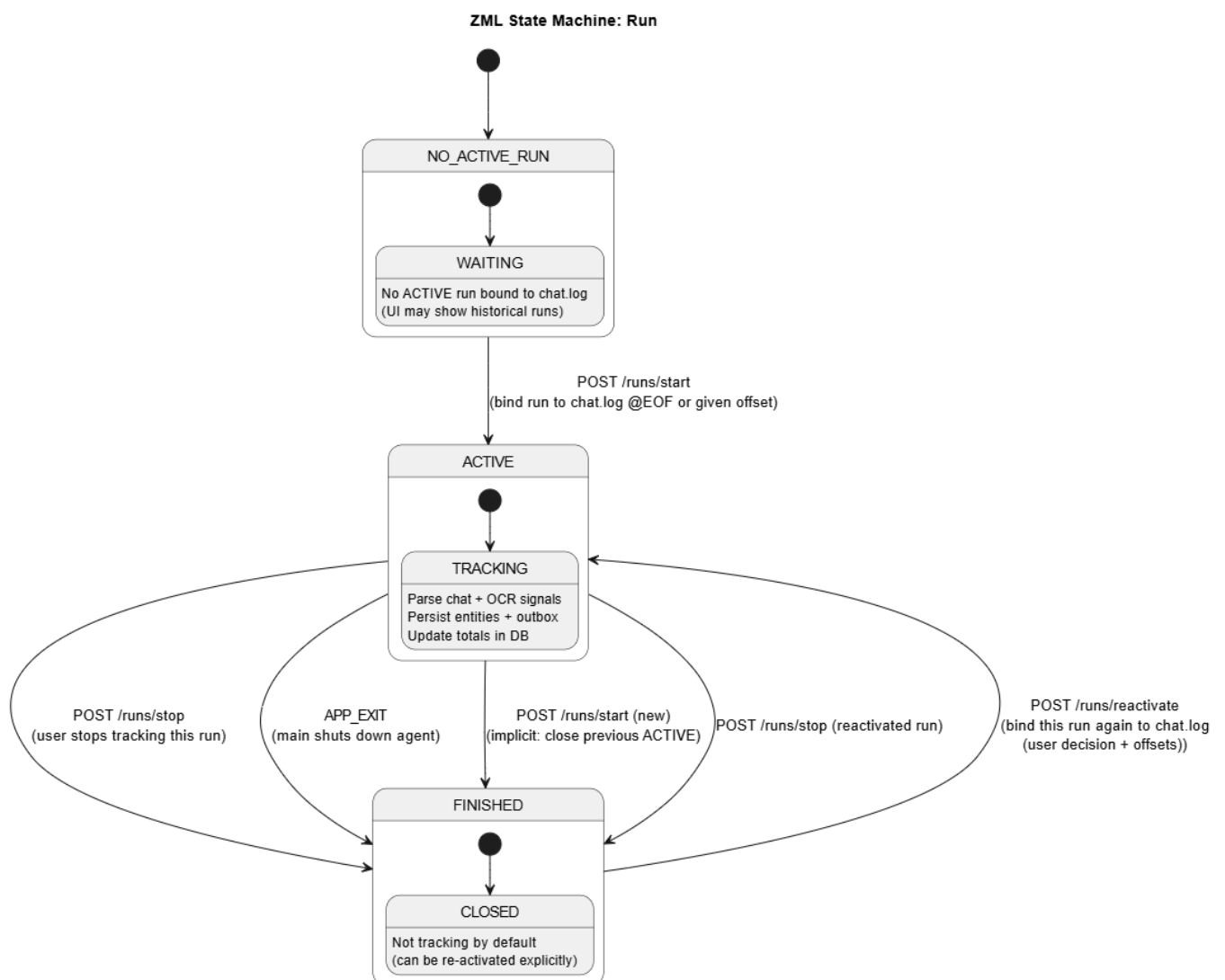
7.4.1 Run lifecycle

Run is a **container for events**; time boundaries are user-controlled.

- **NO_ACTIVE_RUN** → **ACTIVE** (manual Start, or auto-start on first detected drop if enabled)
- **ACTIVE** → **FINISHED** (manual Stop, or Main finishes the run on app exit)

Run totals:

- costs from ProbeDrops (and/or cost segments if enabled)
- returns from LootEvents sum (TT, mPEC)

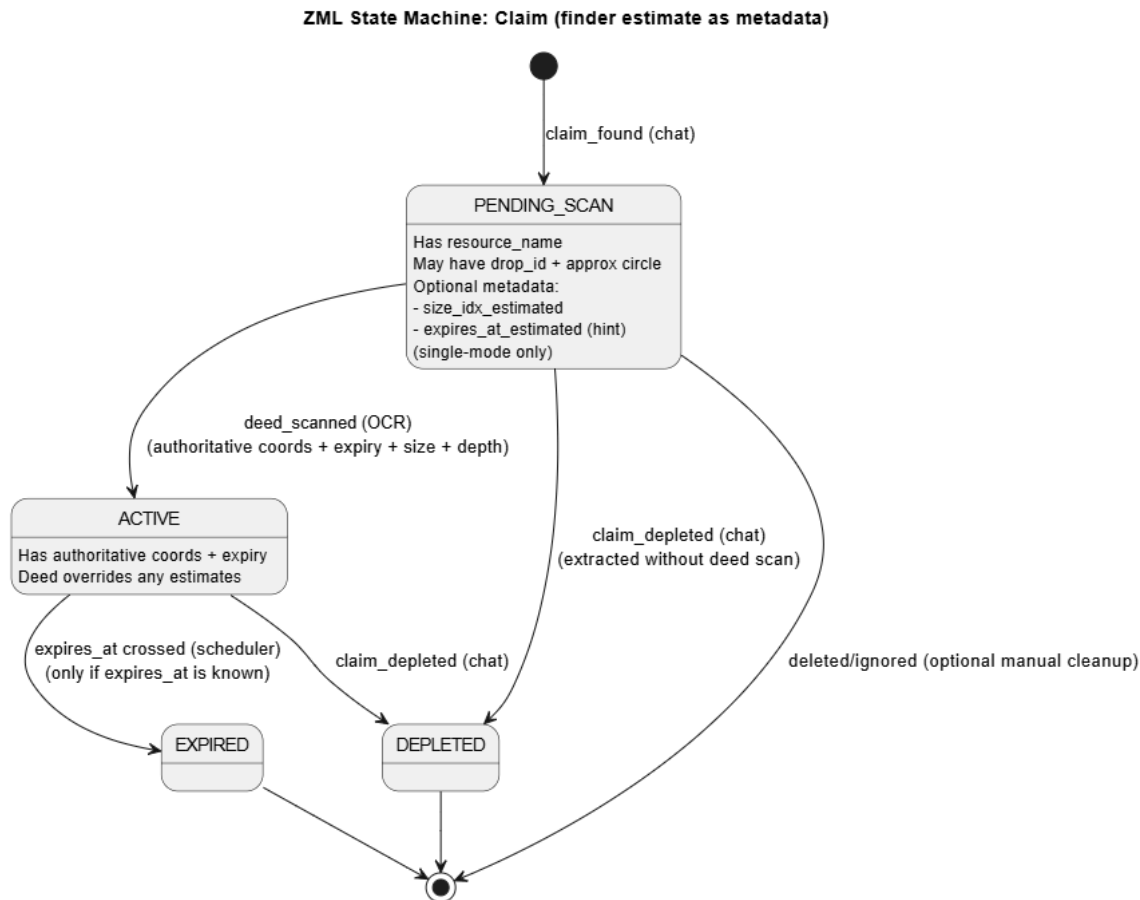


7.4.2 Claim lifecycle

- **PENDING_SCAN** (created from chat)
- **ACTIVE** (after deed scan provides coords + expiry)

Terminal states:

- **DEPLETED** (chat indicates fully extracted)
- **EXPIRED** (expiry exceeded without depletion; only possible if deed scan provided expiry)



7.5 Live UI Rendering Rules (UI Policy)

Rendering policy lives in the UI layer (React), but input data and correlation rules are defined here.

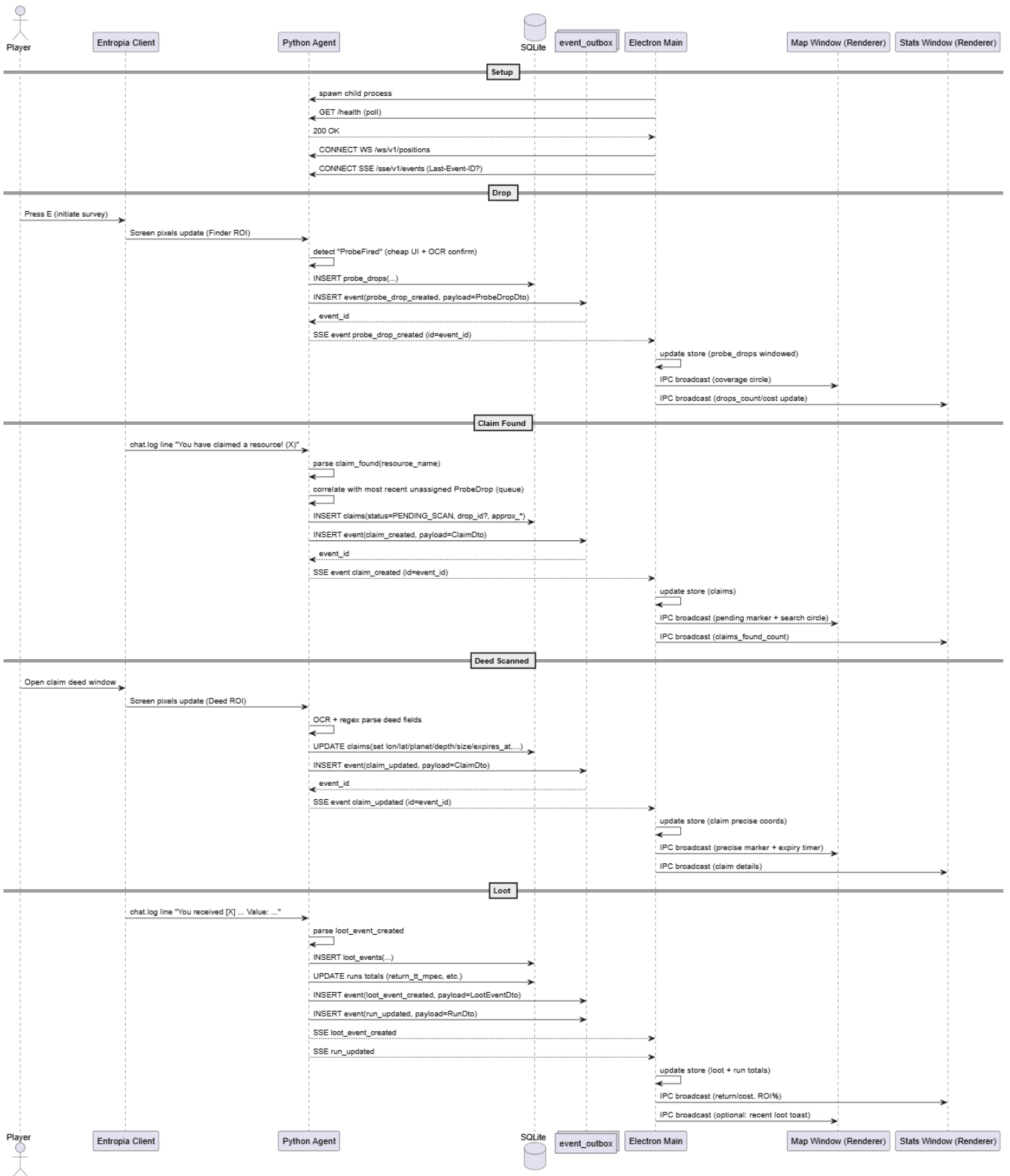
Coverage layer

- Rendered from **ProbeDrop** within a configurable time window (default 30–60 minutes).
- Main broadcasts either:
 - raw **ProbeDrop** batch events, or
 - pre-shaped “coverage primitives” (circle list) depending on performance.

Claims

- **PENDING_SCAN** : fuzzy marker / search circle based on **approx_center + approx_radius_m** if available
- **ACTIVE** : precise marker at authoritative claim coordinates
- Terminal states: visually distinct (dim/archived), optionally hidden by default filters

ZML Sequence: ProbeFired -> ProbeDrop -> ClaimFound -> DeedScanned -> UI



8. API & DTO Contracts (REST + WebSocket + SSE)

8. API & DTO Contracts (REST + WebSocket + SSE)

This page defines the external contracts exposed by the local Python backend (“Agent”) for the desktop app.

Electron Main is the only supported client of the Agent API.

Renderers (React windows) do not call the Agent directly and communicate with Electron Main over IPC only.

8.1 Principles

- **Local-only:** binds to 127.0.0.1 by default.
 - **Single client:** only Electron Main is expected to connect to the Agent.
 - **No auth in v2.0:** optional API key can be added later (still local-only).
 - **Versioned endpoints**
 - REST: `/api/v1`
 - WebSocket (positions): `/ws/v1`
 - SSE (domain events): `/sse/v1`
 - **Money precision:** all monetary values use `*_mpec` integers (milli-PEC).
 - **Timestamps:** ISO 8601 UTC strings in DTOs (`YYYY-MM-DDTHH:MM:SS.sssZ`).
DB may store epoch-ms; the API always returns ISO.
 - **IDs:** opaque strings (UUID recommended).
 - **Forward-compatible payloads:** event payloads may add fields without breaking older clients.
 - **Deterministic resume:** SSE supports resume via `Last-Event-ID` backed by a persisted `event_outbox`.
-

8.2 Network & Ports

Default bind: `127.0.0.1`

Default port: configurable (suggested default: `17877`)

Behavior:

- If the port is taken, the Agent fails fast with a clear log message.
- Electron Main reads port from config/env and connects accordingly.

Base URLs:

- REST: `<http://127.0.0.1:17877/api/v1 >`
 - WS (positions): `ws://127.0.0.1:17877/ws/v1/positions`
 - SSE (events): `<http://127.0.0.1:17877/sse/v1/events >`
-

8.3 Error Format (REST)

All non-2xx responses return:

```
1 {
2   "error": {
3     "code": "VALIDATION_ERROR",
4     "message": "human readable summary",
5     "details": {
6       "field": "explanation"
7     }
8   }
9 }
```

Common error codes:

- `VALIDATION_ERROR` (400)
 - `NOT_FOUND` (404)
 - `CONFLICT` (409)
 - `INTERNAL_ERROR` (500)
-

8.4 WebSocket: Positions Stream (WS)

8.4.1 Contract

WS is used only for high-frequency player positions. It is best-effort and does not support replay.

Endpoint: `/ws/v1/positions`

Message: plain JSON (no envelope required)

Example:

```
1 {
2   "ts": "2026-01-08T08:15:30.123Z",
3   "planet_code": "CYRENE",
4   "lon": 137448,
5   "lat": 78413,
6   "source": "COMPASS_OCR"
7 }
```

Notes:

- On reconnect, Electron Main heals state via REST + SSE resume.
- If position replay is ever needed, add it as a bounded REST query (not WS history).

- Throttling/smoothing for UI is Main responsibility.

8.5 Server-Sent Events: Domain Event Stream (SSE)

SSE carries all non-position domain events (drops, claims, loot, run updates, segment edits). It supports resume using `Last-Event-ID`.

Endpoint: `/sse/v1/events`

SSE frame format:

```
1 id: <event_id>
2 event: <type>
3 data: <json>
```

Example:

```
1 id: 12345
2 event: probe_drop_created
3 data: {"schema_version":1,"id":12345,"type":"probe_drop_created","ts":"2026-01-08T08:15:30.123Z",
```

8.5.1 Event ID and Resume

Requirement: `id` must be monotonically increasing and persistent so that `Last-Event-ID` works across restarts.

Implementation guideline:

- SSE is backed by `event_outbox` table.
- `id` equals `event_outbox.id` (`AUTOINCREMENT` integer).
- Agent streams events in ascending id order.

8.5.2 Event Envelope (SSE data payload)

All SSE `data` values are JSON in a stable envelope:

```
1 {
2   "schema_version": 1,
3   "id": 12345,
4   "type": "probe_drop_created",
5   "ts": "2026-01-08T08:15:30.123Z",
6   "run_id": "run_7b3c...",
7   "payload": {}
8 }
```

Fields:

- `schema_version` : envelope version
- `id` : stable event id (DB-backed)
- `type` : event name

- `ts` : event timestamp (UTC)
- `run_id` : optional
- `payload` : event-specific DTO (or a small “refetch hint” for large objects)

8.5.3 Event Types (v2.0 MVP)

- `probe_drop_created` → `ProbeDropDto`
- `claim_created` → `ClaimDto` (usually `PENDING_SCAN`)
- `claim_updated` → `ClaimDto`
- `loot_event_created` → `LootEventDto`
- `run_updated` → `RunDto`
- `cost_segments_updated` → `{ "run_id": "...", "segments_version": 12 }`
- (optional) `config_updated` → `{ "what": "tool_profiles|constants|hud_calibration"`
`}`

Notes:

- SSE is the primary live stream.
- REST is for snapshot, query, and cold start recovery.
- Segment changes must be followed by `run_updated` (Agent recomputes totals).

8.6 State Recovery Rules (Electron Main responsibility)

On startup or after reconnect, Electron Main restores state via:

1. Connect SSE with `Last-Event-ID` if available.
2. If first start or state uncertain:
 - `GET /runs/current`
 - if active run exists:
 - `GET /runs/{id}/probe-drops?since=...`
 - `GET /runs/{id}/claims?since=...` (optional; add if implemented)
 - `GET /runs/{id}/loot-events?since=...`
 - `GET /runs/{id}/cost-segments`

Memory policy (Main):

- Keep only the last 30–60 minutes of high-frequency objects (configurable).
- Full history remains in SQLite and is queryable via REST.

Gap policy:

- If Main requests `Last-Event-ID > X` but events are no longer available, Main falls back to REST snapshot and resets cursor.

8.7 DTOs (JSON)

8.7.1 Common Types

```
1 { "lon": 137448, "lat": 78413 }
```

`lon`, `lat` : integers as shown by the compass.

8.7.2 Bitmask Conventions (Mining Modes)

`modes_mask` bit values:

- `ORE = 1`
- `ENMATTER = 2`
- `TREASURE = 4`

Examples:

- `1 = ORE`
- `2 = ENMATTER`
- `3 = ORE + ENMATTER`
- `7 = ORE + ENMATTER + TREASURE`

8.7.3 PlayerPositionDto

```
1 {  
2   "ts": "2026-01-08T08:15:30.123Z",  
3   "planet_code": "CYRENE",  
4   "lon": 137448,  
5   "lat": 78413,  
6   "source": "COMPASS_OCR"  
7 }
```

8.7.4 RunDto

```
1 {  
2   "id": "run_7b3c...",  
3   "state": "ACTIVE",  
4   "started_at": "2026-01-08T08:00:00.000Z",  
5   "ended_at": null,  
6  
7   "planet_code": "CYRENE",  
8  
9   "modes_mask_last": 3,  
10  
11   "total_cost_tt_mpec": 1234500,  
12   "total_cost_paid_mpec": 1250000,  
13  
14   "total_return_tt_mpec": 678900,  
15   "total_return_paid_mpec": 678900,  
16 }
```

```

16
17     "drops_count": 300,
18     "claims_found_count": 12,
19
20     "cost_segments_version": 12
21 }

```

Notes:

- `modes_mask_last` is UI-friendly “last observed” mask for filtering; it is not authoritative per-drop.
- `cost_segments_version` increments whenever segments are replaced.
- Chat binding fields belong in a diagnostics DTO (see `RunDiagnosticsDto`) to avoid bloating the main contract.

8.7.5 ProbeDropDto

```

1  {
2      "id": "drop_1a2b...",
3      "run_id": "run_7b3c...",
4      "ts": "2026-01-08T08:15:30.123Z",
5
6      "planet_code": "CYRENE",
7      "lon": 137448,
8      "lat": 78413,
9
10     "radius_m": 55.0,
11     "modes_mask": 3,
12
13     "probes_per_drop": 1,
14     "ammo_per_drop": 0,
15
16     "cost_tt_mpec": 5000,
17     "cost_paid_mpec": 5000
18 }

```

Notes:

- `modes_mask` is snapshotted from the latest stable finder state at the moment `probe_fired` is detected.
- Cost scaling rules must be explicit in tool profile / cost logic.

8.7.6 ClaimDto

```

1  {
2      "id": "claim_c91e...",
3      "run_id": "run_7b3c...",
4
5      "status": "PENDING_SCAN",
6      "category": "ORE",
7
8      "found_at": "2026-01-08T08:16:10.000Z",
9
10     "resource_name": "Yellow Crystal",
11     "resource_code": "YELLOW_CRYSTAL",
12
13     "size_label": "Minimal",
14     "size_idx": 1,
15     "depth_m": 19,
16 }

```

```

17  "planet_code": null,
18  "lon": null,
19  "lat": null,
20  "altitude": null,
21
22  "expires_at": null,
23
24  "drop_id": "drop_1a2b...",
25  "approx_center": { "lon": 137448, "lat": 78413 },
26  "approx_radius_m": 55.0
27 }

```

Notes:

- `category` is best-effort classification derived from resource mapping (YAML). Unknown → `"UNDEFINED"`.
- Before deed scan: coords/expiry may be null; claim is shown via approximate anchor.
- `expires_at` is authoritative when set (from deed scan). Any hint expiry is optional and separate.

8.7.7 LootEventDto

```

1  {
2    "id": "loot_55aa...",
3    "run_id": "run_7b3c...",
4    "ts": "2026-01-08T08:40:00.000Z",
5
6    "resource_name": "Yellow Crystal",
7    "resource_code": "YELLOW_CRYSTAL",
8
9    "quantity": 15,
10   "value_tt_mpec": 1500
11 }

```

8.7.8 ToolProfileDto (config)

```

1  {
2    "id": "rookie_terraster",
3    "display_name": "Rookie TerraMaster",
4    "radius_m": 55.0,
5
6    "probe_tt_mpec_per_drop": 5000,
7    "probe_mu_bps": 10000,
8
9    "ammo_tt_mpec_per_unit": 100,
10   "ammo_mu_bps": 10000,
11
12   "finder_decay_tt_mpec_per_drop_per_mode": 239,
13   "finder_decay_mu_bps": 10100,
14
15   "extractor_decay_tt_mpec_per_tick": 120,
16   "extractor_decay_mu_bps": 10000
17 }

```

8.7.9 RunCostSegmentDto

Segments are the UI-editable layer that explains and overrides how the run costs are computed without rewriting probe drop history.

```

1  {

```

```

2  "id": "seg_01...",
3  "run_id": "run_7b3c...",
4  "sort_order": 10,
5
6  "label": "Amp A (ORE+ENM)",
7  "note": null,
8
9  "applies_to": "PROBE_DROPS",
10 "from_drop_index": 1,
11 "to_drop_index": 150,
12
13 "modes_mask_filter": 3,
14
15 "amp_tt_mpec_per_drop": 2500,
16 "amp_mu_bps": 12000,
17
18 "flat_tt_mpec": 0,
19 "flat_mu_bps": 10000,
20
21 "created_at": "2026-01-08T08:45:00.000Z",
22 "updated_at": "2026-01-08T08:45:00.000Z"
23 }

```

Rules:

- Segments are ordered by `sort_order` and must form a valid partition for the run (Agent validates).
- Segment edits do not mutate `probe_drops` ; they affect recomputed totals only.
- After successful replace:
 - emit `cost_segments_updated` (SSE)
 - recompute totals
 - emit `run_updated` (SSE)

8.7.10 RunDiagnosticsDto (optional)

Diagnostics are useful for troubleshooting but not required for normal UI operation:

```

1  {
2    "run_id": "run_7b3c...",
3    "chat_file_id": "inode:12345|size:987654|mtime:...",
4    "chat_start_offset": 12345678,
5    "chat_last_offset": 12400999,
6    "chat_last_ts": "2026-01-08T08:40:00.000Z"
7  }

```

8.8 REST Endpoints (v2.0 MVP)

8.8.1 Health & Meta

- `GET /health` → 200 { "status": "ok" }
- `GET /meta` → agent version, schema version, config info (no secrets)

8.8.2 Runs

- `GET /runs/current` → `RunDto` or 404

- GET /runs (query: from, to, limit, cursor) → { items, next_cursor }
- GET /runs/{run_id} → RunDto

Optional controls:

- POST /runs/start
- POST /runs/stop

Optional diagnostics:

- GET /runs/{run_id}/diagnostics → RunDiagnosticsDto

8.8.3 Probe Drops

- GET /runs/{run_id}/probe-drops (query: since, until) → { items: ProbeDropDto[] }

8.8.4 Claims

- GET /runs/{run_id}/claims → { items: ClaimDto[] }
- GET /claims/{claim_id} → ClaimDto

Optional helpers:

- POST /claims/{claim_id}/mark-depleted
- POST /claims/{claim_id}/mark-expired

8.8.5 Loot

- GET /runs/{run_id}/loot-events → { items: LootEventDto[] }

8.8.6 Cost Segments

- GET /runs/{run_id}/cost-segments → { items: RunCostSegmentDto[] }
- PUT /runs/{run_id}/cost-segments → { items: RunCostSegmentDto[] }

PUT semantics:

- replaces all segments for the run in a single transaction
- validates partition and ordering rules
- on success emits:
 - cost_segments_updated (SSE)
 - run_updated (SSE) after recompute

8.8.7 Config

- GET /config/tool-profiles
- PUT /config/tool-profiles (replace validated set)

- GET /config/constants
- PUT /config/constants

Optional (HUD wizard):

- GET /config/hud-calibration
- PUT /config/hud-calibration

8.9 Money & MU Precision (TT vs Paid)

(No changes.)

8.10 CORS / Electron Notes

Agent API is consumed only by Electron Main.
CORS can be disabled or minimal by default.

8.11 Versioning & Compatibility

Breaking changes increment: /api/v{n} , /ws/v{n} , /sse/v{n}

schema_version changes only when the event envelope changes; payloads evolve by adding fields.

Contracts v1

Contracts v1 — Single Source of Truth (Agent ↔ Main ↔ Renderers)

0) Scope & principles

- **Python Agent** exposes local API for **exactly one client: Electron Main**. Renderers **never** talk to Agent directly.
 - Streaming:
 - **WS** only for high-frequency positions (**best-effort, no replay**).
 - **SSE** for all other domain events with **resume via Last-Event-ID** backed by `event_outbox`.
 - **REST** for snapshots, history queries, and config writes.
-

1) Versioning & compatibility

- Breaking changes bump path version:
 - `/api/v{n}`, `/ws/v{n}`, `/sse/v{n}`
 - **SSE envelope** has `schema_version` which changes only when the envelope changes; payloads evolve **additively**.
 - IPC payloads include `ipc_version: 1`; only additive changes without bump.
-

2) Money, MU, and bitmasks (conventions)

- Money fields: `*_mpec` are **INTEGER milli-PEC**. MU: `*_mu_bps` are **basis points** (e.g. `10000` = 1.00x).
 - Mining modes bitmask (`modes_mask`):
 - `ORE = 1`, `ENMATTER = 2`, `TREASURE = 4`
-

3) REST API (Agent ↔ Main)

Base

- Base path: `/api/v1` (treat examples without prefix in older notes as shorthand).

3.1 Health & Meta

- `GET /api/v1/health` → `{ status: "ok" }`
- `GET /api/v1/meta` → agent version, schema version, config info (no secrets).

3.2 Runs

- GET /api/v1/runs/current → RunDto or 404
- GET /api/v1/runs (query: from, to, limit, cursor) → { items, next_cursor }
- GET /api/v1/runs/{run_id} → RunDto
- Optional controls:
 - POST /api/v1/runs/start
 - POST /api/v1/runs/stop
- Optional diagnostics:
 - GET /api/v1/runs/{run_id}/diagnostics → RunDiagnosticsDto

3.3 Probe drops

- GET /api/v1/runs/{run_id}/probe-drops (query: since, until) → { items: ProbeDropDto[] }

3.4 Claims

- GET /api/v1/runs/{run_id}/claims → { items: ClaimDto[] }
- GET /api/v1/claims/{claim_id} → ClaimDto
- Optional helpers:
 - POST /api/v1/claims/{claim_id}/mark-depleted
 - POST /api/v1/claims/{claim_id}/mark-expired

3.5 Loot

- GET /api/v1/runs/{run_id}/loot-events → { items: LootEventDto[] }

3.6 Cost segments (editable cost model)

- GET /api/v1/runs/{run_id}/cost-segments → { items: RunCostSegmentDto[] }
- PUT /api/v1/runs/{run_id}/cost-segments → { items: RunCostSegmentDto[] }

Semantics (MVP):

- **replaces all segments** in a **single transaction**
- validates partition/ordering rules
- on success emits (SSE): `cost_segments_updated`, then `run_updated` (after totals recompute)

3.7 Config

- GET /api/v1/config/tool-profiles
- PUT /api/v1/config/tool-profiles (replace validated set)
- GET /api/v1/config/constants
- PUT /api/v1/config/constants
- Optional HUD wizard:
 - GET /api/v1/config/hud-calibration
 - PUT /api/v1/config/hud-calibration

3.8 Shutdown

- Preferred: POST /api/v1/shutdown or SIGTERM (lifecycle owned by Main).

4) WebSocket (Agent ↔ Main): positions

- Endpoint: GET /ws/v1/positions
- Payload: **plain JSON (no envelope)**, best-effort, no replay.
- Recovery: On reconnect, Main heals via REST snapshot + SSE resume.

PlayerPositionDto (WS message)

Fields:

- ts (ISO-8601 UTC)
- planet_code
- lon (int)
- lat (int)
- source (e.g. COMPASS_OCR)

5) SSE (Agent ↔ Main): domain event stream

5.1 Endpoint & resume

- Endpoint: GET /sse/v1/events
- Resume: Last-Event-ID header supported; event ids are **monotonic DB-backed**: event_outbox.id (AUTOINCREMENT).

5.2 SSE envelope (stable)

Every SSE data payload uses a stable envelope:

- schema_version

- `id` (stable event id, DB-backed)
- `type` (event name)
- `ts` (UTC timestamp)
- `run_id` (optional)
- `payload` (DTO or small “refetch hint”)

5.3 Event types (v2.0 MVP)

- `probe_drop_created` → `ProbeDropDto`
- `claim_created` → `ClaimDto` (usually `PENDING_SCAN`)
- `claim_updated` → `ClaimDto`
- `loot_event_created` → `LootEventDto`
- `run_updated` → `RunDto`
- `cost_segments_updated` → `{ run_id, segments_version }`
- optional `config_updated` → `{ what: "tool_profiles|constants|hud_calibration" }`

Rule: segment changes must be followed by `run_updated` (Agent recomputes totals).

5.4 Gap policy (outbox retention)

If Main requests `Last-Event-ID` that is no longer available (pruned/retention), Main must:

- fall back to REST snapshot
- reset cursor

6) IPC (Renderer ↔ Electron Main)

6.1 Rules

- All window comms go through IPC `zml/*`.
- Payloads include `ipc_version: 1`.

6.2 Renderer → Main (commands)

- `zml/cmd:get_bootstrap_state` — initial snapshot slice + metadata for a newly opened window
- `zml/cmd:open_window` — opens `main | map | hud_cost` with options (size preset, always-on-top, click-through)
- `zml/cmd:set_window_options` — `always_on_top`, `opacity`, `click_through`, `bounds`

- `zml/cmd:query_history` — runs list, claims list, loot aggregation (paged)
- `zml/cmd:config:update_tool_profiles` — tool profiles edited in UI
- `zml/cmd:config:update_hud_calibration` — calibration result (ROIs + resolution metadata)
- `zml/cmd:resources:reload_yaml` — reload resource mapping YAML (Main/Agent then emits `config_updated`)

6.3 Main → Renderer (push)

- `zml/push:state_patch` — primary batched update channel (MVP can replace full slices; later patches)
- `zml/push:position` — optional separate high-rate channel to Map window
- `zml/push:notice` — user-facing notices (backend disconnected, calibration invalid, modes disabled unexpectedly, deed scanner toggled)
- Also referenced:
 - `zml/push/appReady` (initial “hydrated” signal + initial snapshot)
 - `zml/push/agentStatus` (Agent crash/restart status)

7) Recovery flows (Main-owned)

7.1 Cold start / reconnect (Main responsibility)

On startup or after reconnect:

1. Connect SSE with `Last-Event-ID` if available
2. If first start / state uncertain:
 - `GET /runs/current`
 - if active run exists: fetch deltas via REST:
 - probe-drops, claims (optional), loot-events, cost-segments
3. Positions WS reconnect is best-effort; correctness comes from SSE/REST.

7.2 chat.log offline gap prompt (safe-first)

If an ACTIVE run exists and there’s likely an offline gap, Main must prompt the user:

- Option A: resume and sync missed events (parse from last offset → EOF, run-scoped dedup)
 - Option B: resume without sync (skip offline gap; set offsets to current EOF)
- Default recommendation if unsure: **Option B** (safe-first; no silent backfill).

8) Error model (REST/IPC)

REST error payload (canonical)

- `error.code` (e.g. `VALIDATION_ERROR`)
- `error.message` (human-readable)
- `error.details` (object; field → explanation)

Recommended minimal codes:

- `VALIDATION_ERROR`
 - `NOT_FOUND`
 - `CONFLICT`
 - `INTERNAL_ERROR`
-

9) Key DTOs (external contracts)

Poniżej tylko kluczowe pola, żeby nie dublować całych JSON-ów.

RunDto

- `id`, `state` (`ACTIVE|FINISHED`)
- `started_at`, `ended_at`
- `planet_code`
- `modes_mask_last` (UI-friendly last observed)
- totals:
 - `total_cost_tt_mpec`, `total_cost_paid_mpec`
 - `total_return_tt_mpec`, `total_return_paid_mpec`
- counters: `drops_count`, `claims_found_count`
- `cost_segments_version` (increments when segments replaced)

ProbeDropDto

- `id`, `run_id`, `ts`
- `planet_code`, `lon`, `lat`
- `radius_m`
- `modes_mask` (snapshotted at probe fired)
- `probes_per_drop`, `ammo_per_drop`
- `cost_tt_mpec` (+ paid fields if exposed)

ClaimDto

- `id`, `run_id`
- `status` (e.g. `PENDING_SCAN` ; later `ACTIVE` , terminal `DEPLETED|EXPIRED`)
- `category` (best-effort; unknown → `UNDEFINED`)
- `found_at`
- resource identity:
 - `resource_name` , `resource_code`
- deed-authoritative fields (may be null before scan):
 - `planet_code` , `lon` , `lat` , `altitude`
 - `depth_m` , `size_label` , `size_idx`
 - `expires_at` (authoritative when set)
- correlation fields:
 - `drop_id` (linked probe drop)
 - `approx_center` , `approx_radius_m` (until deed scan)

LootEventDto

- `id` , `run_id` , `ts`
- `resource_name` , `resource_code`
- `quantity`
- `value_tt_mpec`

RunCostSegmentDto

Rules:

- ordered by `sort_order`
- must be a valid partition (no gaps/overlaps)
- edits don't mutate `probe_drops` ; they affect recomputed totals only
- replace emits: `cost_segments_updated` , then `run_updated`

9. Data Model - SQLite

9. Data Model – SQLite Schema (v2.0 MVP)

This page defines the local SQLite data model used by the Python Agent. SQLite is the source of truth for all persisted mining history (runs, drops, claims, loot). High-frequency UI state (live position stream) is not persisted unless captured as snapshots at key events.

9.1 Design Goals

- Local-first and offline.
- Append-first event capture where possible, but entities are stored in query-friendly tables.
- Idempotent writes: repeated parsing of the same log line/event must not duplicate records.
- Fast queries for:
 - run lists + details,
 - recent probe drops (time window),
 - claim history filtered by planet/resource/time/bbox,
 - loot history aggregated by run/resource.
- No raw chat lines stored in analytical tables.

9.2 Conventions

- All money uses `*_mpec INTEGER` (milli-PEC).
- All timestamps stored as `INTEGER epoch_ms` (recommended for indexing). API exposes ISO-8601.
- IDs are `TEXT` (UUID).
- Planet codes are uppercase short strings (`CYRENE` , `ARKADIA` , etc.).
- SQLite: enable `PRAGMA foreign_keys = ON` . Use WAL mode in production.

9.3 Tables (Entities)

9.3.1 runs

Stores a mining session and its totals. Runs are “containers” for events; they are not immutable in the UX sense (you can re-open and adjust), but **only one run can be ACTIVE**.

Column	Type	Constraints / Default	Indexes	Notes
id	TEXT	PK		UUID

state	TEXT	NOT NULL	idx_runs_state (state)	ACTIVE FINISHED
started_at_ms	INTEGER	NOT NULL	idx_runs_started_at (started_at_ms DESC)	epoch ms
ended_at_ms	INTEGER	NULL		epoch ms
planet_code	TEXT	NULL	idx_runs_planet_ti me (planet_code, started_at_ms) (optional)	optional
modes_bitmask_se en	INTEGER	NOT NULL		
total_cost_tt_mpec	INTEGER	NOT NULL DEFAULT 0		running total
total_cost_paid_mp ec	INTEGER	NOT NULL DEFAULT 0		running total
total_return_tt_mpe c	INTEGER	NOT NULL DEFAULT 0		from loot lines
total_return_paid_ mpec	INTEGER	NOT NULL DEFAULT 0		v2.0 = TT
drops_count	INTEGER	NOT NULL DEFAULT 0		derived/incremente d
claims_found_coun t	INTEGER	NOT NULL DEFAULT 0		derived/incremente d
active_tool_profile_ id	TEXT	NULL		optional “default profile” for new drops
created_at_ms	INTEGER	NOT NULL		
updated_at_ms	INTEGER	NOT NULL		

chat_file_id	INTEGER	NOT NULL		identity of chat.log (inode+size+mtime or similar)
chat_start_offset	INTEGER	NOT NULL		offset captured at RunStart (typically EOF)
chat_last_offset	INTEGER	NOT NULL		last processed offset for ACTIVE run (opportunistic updates)
chat_last_ts_ms	INTEGER	NOT NULL		timestamp of the last parsed chat line persisted for this run

Rules (application-level):

- At most one `runs.state = 'ACTIVE'` at a time (enforce in app; optional UNIQUE partial index if you want).
- On **RunStart**: set `chat_file_id`, set `chat_start_offset` to EOF, set `chat_cursor_offset = chat_start_offset`.
- On restart:
 - if ACTIVE run exists → resume from `chat_cursor_offset` if `chat_file_id` matches; else fall back to `chat_start_offset` (or ask user to restart run).
 - if no ACTIVE run → jump to EOF and wait for user to Start.

9.3.2 probe_drops

One record per probe drop attempt (coverage anchor + cost input).

Column	Type	Constraints / Default	Indexes	Notes
id	TEXT	PK		UUID
run_id	TEXT	NOT NULL FK	idx_probe_drops_run_ts (run_id,	→ runs(id)

			ts_ms)	
ts_ms	INTEGER	NOT NULL	idx_probe_drops_ts (ts_ms)	drop timestamp
planet_code	TEXT	NULL	idx_probe_drops_b box (planet_code, lon, lat) (optional)	
lon	INTEGER	NOT NULL	idx_probe_drops_b box (planet_code, lon, lat) (optional)	
lat	INTEGER	NOT NULL	idx_probe_drops_b box (planet_code, lon, lat) (optional)	
radius_m	REAL	NOT NULL		from tool profile
modes_bitmask	INTEGER	NOT NULL	idx_probe_drops_m odes (modes_bitmask) (optional)	fixes “MIXED says nothing” (see below)
probes_per_drop	INTEGER	NOT NULL		from finder UI
ammo_per_drop	INTEGER	NOT NULL		from finder UI
cost_tt_mpec	INTEGER	NOT NULL		computed at insert
cost_paid_mpec	INTEGER	NOT NULL		computed at insert
tool_profile_id	TEXT	NULL	idx_probe_drops_p rofile (tool_profile_id) (optional)	snapshot link
assigned_claim_id	TEXT	NULL	idx_probe_drops_a ssigned (assigned_claim_id) (optional)	correlation helper
created_at_ms	INTEGER	NOT NULL		

drop_index	INTEGER	NOT NULL	ux_probe_drops_run_index (run_id, drop_index) UNIQUE	Monotonic per run (0..N-1). Stable addressing for ranges.
------------	---------	----------	---	---

`modes_bitmask` **recommendation:**

- bit 0 = ORE
- bit 1 = ENMATTER
- bit 2 = TREASURE

Examples:

- `1` = ORE
- `2` = ENMATTER
- `3` = ORE+ENMATTER (your “mixed”)
- `7` = all three

This lets you compute costs deterministically and avoids “MIXED” being a black box.

9.3.3 `claims`

Claim entity (created from chat; enriched by deed scan).

Column	Type	Constraints / Default	Indexes	Notes
id	TEXT	PK		UUID
run_id	TEXT	NOT NULL FK	idx_claims_run_found (run_id, found_at_ms)	→ runs(id)
status	TEXT	NOT NULL	idx_claims_status (status)	PENDING_SCAN ACTIVE DEPLETED EXPIRED
category	TEXT	NOT NULL DEFAULT 'UNKNOWN'		

found_at_ms	INTEGER	NOT NULL	idx_claims_planet_time (planet_code, found_at_ms)	chat time
resource_name	TEXT	NOT NULL		raw display name
resource_code	TEXT	NULL	idx_claims_resource (resource_code)	normalized
size_label	TEXT	NULL		from deed (authoritative)
size_idx	INTEGER	NULL		from deed (authoritative)
depth_m	INTEGER	NULL		from deed
planet_code	TEXT	NULL	idx_claims_bbox (planet_code, lon, lat) (optional)	from deed
lon	INTEGER	NULL	idx_claims_bbox (planet_code, lon, lat) (optional)	from deed
lat	INTEGER	NULL	idx_claims_bbox (planet_code, lon, lat) (optional)	from deed
altitude	INTEGER	NULL		from deed
expires_at_ms	INTEGER	NULL	idx_claims_expires (expires_at_ms) (optional)	from deed
drop_id	TEXT	NULL FK	idx_claims_drop (drop_id) (optional)	→ probe_drops(id)
approx_lon	INTEGER	NULL		anchored to drop
approx_lat	INTEGER	NULL		anchored to drop
approx_radius_m	REAL	NULL		anchored to drop

finder_size_label	TEXT	NULL		Early size hint from Finder UI (if enabled). Not authoritative.
finder_size_idx	INTEGER	NULL		Early size hint index (if enabled). Not authoritative.
finder_expires_at_est_ms	INTEGER	NULL		Estimated expiry derived from TTL-by-size table (single-mode only). Deed scan overrides.
finder_prescan_at_ms	INTEGER	NULL		When the finder pre-scan hint was captured.
deed_scanned_at_ms	INTEGER	NULL		when deed was read
depleted_at_ms	INTEGER	NULL		from chat
claim_fingerprint	TEXT	NULL	ux_claims_fingerprint (claim_fingerprint) (optional UNIQUE)	future sync dedup
created_at_ms	INTEGER	NOT NULL		
updated_at_ms	INTEGER	NOT NULL		

Application rules (not DB constraints):

- `status != PENDING_SCAN ⇒ planet_code/lon/lat` should be non-null.
- `expires_at_ms != NULL ⇒ deed_scanned_at_ms != NULL`.
- `expires_at_ms` from deed stays authoritative. Finder fields are purely hints and can be ignored.

`claim_fingerprint` (checksum) suggestion (only when deed scan exists):

- Build canonical string:

- `planet_code|lon|lat|resource_code|size_idx|depth_m`
- Hash: `sha1` or `blake2s` (store hex).
- Why: stable dedup when syncing across clients later, even if local IDs differ.

9.3.4 `loot_events`

Atomic loot record parsed from chat log.

Column	Type	Constraints / Default	Indexes	Notes
<code>id</code>	TEXT	PK		UUID
<code>run_id</code>	TEXT	NOT NULL FK	<code>idx_loot_run_ts</code> (<code>run_id</code> , <code>ts_ms</code>)	→ <code>runs(id)</code>
<code>ts_ms</code>	INTEGER	NOT NULL	<code>idx_loot_ts</code> (<code>ts_ms</code>)	
<code>resource_name</code>	TEXT	NOT NULL		from chat
<code>resource_code</code>	TEXT	NULL	<code>idx_loot_resource</code> (<code>resource_code</code>)	normalized
<code>quantity</code>	INTEGER	NOT NULL		from chat
<code>value_tt_mpec</code>	INTEGER	NOT NULL		from chat
<code>created_at_ms</code>	INTEGER	NOT NULL		

Notes:

- No claim attribution in v2.0 MVP.
- Run totals are derived/updated incrementally from these events.
- If you want idempotency on chat restarts: see `event_outbox.dedup_key` below (recommended), not a separate “loot dedup” mechanism.

9.4 Tables (Config / Calibration)

9.4.1 `tool_profiles`

User-managed tool profiles used for cost computation and coverage radius.

Column	Type	Constraints / Default	Indexes	Notes
id	TEXT	PK		
display_name	TEXT	NOT NULL	idx_tool_profiles_name (display_name)	
radius_m	REAL	NOT NULL		
probe_tt_mpec_per_drop	INTEGER	NOT NULL		
probe_mu_bps	INTEGER	NOT NULL DEFAULT 10000		
ammo_tt_mpec_per_unit	INTEGER	NOT NULL		
ammo_mu_bps	INTEGER	NOT NULL DEFAULT 10000		
finder_decay_tt_mpec_per_drop_per_mode	INTEGER	NOT NULL		per active mode
finder_decay_mu_bps	INTEGER	NOT NULL DEFAULT 10000		
extractor_decay_tt_mpec_per_tick	INTEGER	NOT NULL		
extractor_decay_mu_bps	INTEGER	NOT NULL DEFAULT 10000		
version	INTEGER	NOT NULL DEFAULT 1	idx_tool_profiles_id_ver (id, version) (optional)	optional versioning
created_at_ms	INTEGER	NOT NULL		
updated_at_ms	INTEGER	NOT NULL		

Notes:

- If you do profile versioning “seriously”: make `(id, version)` the key, and store a stable `profile_key` separately. For MVP, a single `version` integer is enough to track edits.

9.4.2 hud_calibration

Stores calibrated rectangles for OCR regions. One active set per “profile”.

Column	Type	Constraints / Default	Indexes	Notes
id	TEXT	PK		
profile_name	TEXT	NOT NULL	ux_hud_profile (profile_name) (optional UNIQUE)	e.g. <code>default</code>
game_resolution_w	INTEGER	NOT NULL		
game_resolution_h	INTEGER	NOT NULL		
compass_region_json	TEXT	NOT NULL		<code>{"x":..., "y":..., "w":..., "h":...}</code>
finder_region_json	TEXT	NOT NULL		
deed_region_json	TEXT	NULL		
created_at_ms	INTEGER	NOT NULL		
updated_at_ms	INTEGER	NOT NULL		

9.4.3 constants

Global constants that almost never change (but allow override).

Column	Type	Constraints / Default	Indexes	Notes
key	TEXT	PK		

value_json	TEXT	NOT NULL		
updated_at_ms	INTEGER	NOT NULL		

Example keys:

- `probe_tt_mpec_default`
- `ammo_tt_mpec_default`
- `schema_version`

9.5 Idempotency, Chat Cursor & Restarts

9.5.1 Per-run chat cursor (MVP approach)

We **do not** store raw chat lines.

We store **per ACTIVE run**:

- `runs.chat_file_id`
- `runs.chat_start_offset`
- `runs.chat_cursor_offset` (checkpoint)

Runtime:

- parser reads only `[chat_cursor_offset .. EOF]`
- after committing *any* DB batch (drops/claims/loot/outbox), Agent can update `chat_cursor_offset` **opportunistically** (no separate “write loop” only for cursor).

Restart:

- if ACTIVE run exists:
 - if `chat_file_id` matches current file → resume from `chat_cursor_offset`
 - else (rotated/cleared) → fall back to `chat_start_offset` or require user to restart run (choose strictness; MVP can be strict)
- if no ACTIVE run → jump to EOF (user starts a run to begin counting)

This matches your rule: “liczymy tylko od momentu user nacisnął Start”.

9.5.2 Why not a separate `chat_log_state` table in MVP?

You *can* add it later for robustness, but with your UX (explicit Start), the `runs.*chat_*` fields are enough and simpler.

9.6 Recommended: `event_outbox` (SSE backbone)

This makes SSE resume clean and avoids “which table produced what”.

Column	Type	Constraints / Default	Indexes	Notes
id	INTEGER	PK AUTOINCREMENT		SSE Last-Event-ID
ts_ms	INTEGER	NOT NULL	idx_outbox_ts (ts_ms)	event time
type	TEXT	NOT NULL	idx_outbox_type (type) (optional)	e.g. <code>claim_created</code>
run_id	TEXT	NULL	idx_outbox_run (run_id)	filter per run
entity_type	TEXT	NULL	idx_outbox_entity (entity_type, entity_id) (optional)	e.g. <code>claim</code>
entity_id	TEXT	NULL	idx_outbox_entity (entity_type, entity_id) (optional)	
payload_json	TEXT	NOT NULL		full envelope/payload
dedup_key	TEXT	NULL	ux_outbox_dedup (dedup_key) (optional UNIQUE)	idempotency
created_at_ms	INTEGER	NOT NULL		insert time

Notes:

- SSE `/sse/v1/events` streams `event_outbox` rows in order.
- `Last-Event-ID` maps directly to `event_outbox.id`.
- This is not only “connection broke”: it also gives you replay/debug and can help with safe restarts.
- Recommended `dedup_key` patterns (only where it matters):

- chat-derived: `chat:<chat_file_id>:<line_offset>` (or `<ts_ms>:<normalized_line_hash>` if offset is hard)
- OCR-derived: `ocr:<run_id>:<event_type>:<ts_bucket>:<roi_hash>` (best effort)

9.7 Query Patterns (what we optimize for)

- Runs list: `runs ORDER BY started_at_ms DESC LIMIT ?`
- Active run: `runs WHERE state='ACTIVE' LIMIT 1`
- Recent coverage: `probe_drops WHERE ts_ms > now_ms - window_ms`
- Claims history: filter on `(planet_code, found_at_ms)` and `resource_code`, optional bbox index
- Loot aggregation: `GROUP BY resource_code` per run (SQL or Agent-side)

9.8 run_cost_segments (NEW — editable cost model)

This table enables “segment editing” (e.g. “last 50 drops were ore-only”, “amp changed after 200 drops”) **without rewriting** `probe_drops` rows.

Column	Type	Constraints / Default	Indexes	Notes
id	TEXT	PK		UUID.
run_id	TEXT	NOT NULL FK → runs(id)	idx_segments_run_order (run_id, sort_order)	Segment belongs to a single run.
sort_order	INTEGER	NOT NULL	idx_segments_run_order (run_id, sort_order)	UI ordering. Keep stable to avoid churn.
applies_to	TEXT	NOT NULL	idx_segments_run_applies (run_id, applies_to) (optional)	<code>DROP_RANGE</code> or <code>RUN_TOTAL</code> .
from_drop_index	INTEGER	NULL	idx_segments_run_range (run_id, from_drop_index, to_drop_index)	Inclusive. Only for <code>DROP_RANGE</code> .

to_drop_index	INTEGER	NULL	idx_segments_run_range (run_id, from_drop_index, to_drop_index)	Inclusive. Only for <code>DROP_RANGE</code> .
modes_bitmask_filter	INTEGER	NOT NULL DEFAULT 0	idx_segments_modes (run_id, modes_bitmask_filter) (optional)	0 = “all modes”. Otherwise only apply when (drop.modes_bitmask & filter) != 0.
component	TEXT	NOT NULL	idx_segments_component (run_id, component)	What the segment represents: <code>AMP</code> , <code>OVERRIDE</code> , (optional later: <code>TOOL_PROFILE_OVERRIDE</code>).
tt_mpec_per_drop	INTEGER	NULL		Per-drop TT delta (only for <code>DROP_RANGE</code>).
paid_mpec_per_drop	INTEGER	NULL		Per-drop paid delta (only for <code>DROP_RANGE</code>).
flat_tt_mpec	INTEGER	NOT NULL DEFAULT 0		Flat TT delta for <code>RUN_TOTAL</code> (or for range if you really want).
flat_paid_mpec	INTEGER	NOT NULL DEFAULT 0		Flat paid delta for <code>RUN_TOTAL</code> (or for range if you really want).
label	TEXT	NULL		User-facing label (e.g. “Amp L5 (last 50)”, “Ore-only correction”).

note	TEXT	NULL		Free text for audit/debug.
created_at_ms	INTEGER	NOT NULL		
updated_at_ms	INTEGER	NOT NULL		

Recommended app-layer rules

- For `DROP_RANGE`: require `from_drop_index` and `to_drop_index` non-null, and `from <= to`.
- For `RUN_TOTAL`: range columns must be null.
- Keep segments **append/edit friendly**; don't enforce overlap constraints in SQL (UI needs flexibility).

Validation rules (application layer):

- For a given run: segments form a contiguous partition over drop IDs.
- `start_drop_id` must be a drop in the same run.
- If `end_drop_id` set: it must be a drop in the same run and must come after start.

How totals are computed

- `runs.total_cost_*` becomes:
`SUM(probe_drops.cost_* ) + SUM(run_cost_segments.*)`
 (where segment deltas apply based on range + optional mode filter).
- This makes “edit last 50 drops” a cheap update (only segments change + recompute totals).

9.8. Run Cost Segments

Run Cost Segments (Editable Cost Model)

Purpose

MiningLog automatically records **ProbeDrops** (each drop attempt) and other domain events. However, real runs often require **manual corrections**:

- Player changes **amp** mid-run.
- Player switches from **ORE+ENMATTER** to **ORE-only**. (This should be handled by OCR, but just in case)
- OCR missed a mode change, or icons auto-disabled (e.g., amp became unusable).
- User wants a fast “fix totals” without editing hundreds of drops.

To support this, we introduce **Run Cost Segments**: a UI-editable layer that defines how costs are computed over a sequence of ProbeDrops, without rewriting history.

Core idea:

- **ProbeDrops stay immutable historical records** (ts, lon/lat, probes/ammo, modes_bitmask_used snapshot).
- **Segments are the editable interpretation layer** used to compute run totals and display costs in the UI.
- Agent stores segments; Electron Main edits them; computed totals are derived from segments + drops.

High-level Rules

1. Segments are **ordered ranges** over ProbeDrops in a run.
2. Every ProbeDrop belongs to **exactly one segment** (by range rule).
3. Segments define **cost parameters** (tool profile, amp decay, MU overrides, etc.).
4. Editing segments **recomputes totals**, but does not rewrite ProbeDrops.

Segment Model

A segment describes “for these drops, use this cost configuration”.

Segment fields (conceptual):

- `id` (UUID)
- `run_id`
- `name` (optional, UI label like “Amp 5 – mixed”, “Ore-only tail”)

- `range_start_drop_ts_ms` (inclusive) OR `range_start_drop_id`
- `range_end_drop_ts_ms` (exclusive) OR `range_end_drop_id` (nullable = “until end”)
- `tool_profile_id` (required)
- `modes_override_bitmask` (optional; if set, overrides drop’s `modes_bitmask_used` for cost calc only)
- `amp_profile_id` (optional; or inline fields like `amp_decay_tt_mpec_per_drop`, `amp_mu_bps`)
- `multiplier` (optional; e.g. if some mechanic scales per mode)
- `notes` (optional)
- auditing: `created_at_ms`, `updated_at_ms`

Recommended Range Representation

Prefer **drop_id-based ranges** over timestamps:

- `start_drop_id` (inclusive)
- `end_drop_id` (exclusive, nullable)

Because:

- stable across clock skew / parsing delays
- deterministic when user “moves last 50 drops” between segments

Cost Computation (How totals are derived)

For each ProbeDrop in a run:

1. Find its segment by range.
2. Determine effective modes:
 - if `segment.modes_override_bitmask != NULL` → use override
 - else use `probe_drop.modes_bitmask_used`
3. Compute drop cost components:
 - probes/ammo TT (from drop values and constants)
 - finder decay (from tool_profile, may scale by modes)
 - optional amp decay (from segment’s amp config)
4. Apply MU basis points deterministically for “paid” totals.
5. Aggregate to run totals:
 - `total_cost_tt_mpec = sum(drop_cost_tt_mpec)`
 - `total_cost_paid_mpec = sum(drop_cost_paid_mpec)`

Important: run totals are derived from drops+segments, not manually edited fields.

Editing UX (UI behavior)

UI shows a “Cost Segments” panel:

- A list of segments in order.
- Each segment shows:
 - drop count
 - time range
 - selected tool profile
 - optional amp config
 - computed totals for that segment

Common edit actions

A) Split segment at a drop

- User selects a drop boundary (e.g. “after this drop”)
- System splits segment into two with same config
- User edits the new segment config

B) Reassign last N drops

- UI action: “Move last 50 drops to new segment”
- Creates a new segment at boundary and assigns range accordingly

C) Change segment config

- Change amp / tool_profile / override modes
- Immediately recompute totals

Constraints

- Segments must cover the entire run drop sequence with no gaps.
 - Segments must not overlap.
 - “Fix invalid segments” action can auto-merge or re-bound segments.
-

API (Agent ↔ Electron Main)

Read segments

```
GET /api/v1/runs/{run_id}/cost-segments  
→ { items: RunCostSegmentDto[] }
```

Replace segments (simple + safe)

```
PUT /api/v1/runs/{run_id}/cost-segments  
Body: { items: RunCostSegmentDto[] }
```

Semantics:

- replaces all segments for the run in one transaction
- validates partition rules
- on success emits:
 - `cost_segments_updated` (SSE)
 - `run_updated` (SSE) after recompute

Why replace instead of patch?

- fewer edge cases
- easier UI (state is authoritative in Main anyway)
- avoids partial invalid states

Optional helper endpoints (later)

- `POST /runs/{id}/cost-segments/split` (at `drop_id`)
- `POST /runs/{id}/cost-segments/move-last` (n drops)

But MVP can do everything via PUT replace.

Eventing (SSE outbox)

Add SSE event type:

- `cost_segments_updated`
 - payload: { `run_id`, `segments`: [...] } (or only metadata + client refetch)

Whenever segments are changed:

- recompute run totals
- emit `run_updated` afterwards (so UI doesn't do its own math unless desired)

Interaction with “Manual Overrides”

If you still want a super-simple knob:

- Add a “Run Adjustments” table later (add/subtract fixed mPEC).

But **prefer segments**: they keep the accounting explainable.

Notes / Edge Cases

Mode capture race / icon auto-disable

Segments can override modes for cost purposes if OCR/icon state got corrupted.

This avoids rewriting probe_drop history.

OCR skipped some drops

Segments still work; drop count may be lower than “real”.

That’s a separate gap; UI can show “drop count confidence”.

Mixing ore/enmatter UI behavior

We store `modes_bitmask_used` per ProbeDrop from the latest stable finder state at the moment we detected ProbeFired.

Even if icons toggle 200ms later, the snapshot is what we use as the default interpretation — but segments can override.

10. Tech decisions & tradeoffs

10. Tech Decisions & Tradeoffs

This page captures key technical decisions in MiningLog 2.0, the alternatives considered, and the tradeoffs we accept for v2.0 MVP.

10.1 Process Topology: 3 processes, single Agent client

Decision

Run MiningLog as three processes:

- **Python Agent:** OCR + chat tail + SQLite + API
- **Electron Main (Host):** owns AppState, streams, throttling, IPC broadcasting
- **React Renderers (windows):** UI only, IPC only

Why

- Multiple windows must feel like one app while staying responsive.
- Renderers must stay dumb; state and recovery logic belongs to Main.
- OCR and log tailing should not share a process with UI.

Tradeoffs

- More moving parts (process management, restart handling).
 - Slightly more plumbing (IPC types, state snapshots).
-

10.2 Communication: WS for positions, SSE for everything else

Decision

- **WebSocket:** `/ws/v1/positions` (high-frequency, best-effort)
- **SSE:** `/sse/v1/events` backed by `event_outbox` (resume via Last-Event-ID)

Why

- Positions are noisy and don't need replay.
- Domain events require correctness and resume across reconnect/crash.
- SSE + outbox gives deterministic ordering and easy recovery.

Tradeoffs

- Two streaming protocols instead of one.
 - Must maintain outbox retention policy (don't prune too aggressively).
-

10.3 Persistence: SQLite as source of truth

Decision

- Use **SQLite** for all persisted mining history.
- Keep “live-only” state (positions) out of DB unless snapshotted at key moments.

Why

- Offline-first, zero infrastructure.
- Query-friendly for runs/claims/loot history.
- Works great with a single-writer pattern.

Tradeoffs

- SQLite prefers a single writer → we enforce a writer queue.
 - “Spatial queries” are basic (bbox indexes, no real GIS).
-

10.4 Money arithmetic: integer mPEC + basis points MU

Decision

- All monetary values stored as **INTEGER mPEC** (`*_mpec`).
- MU stored as **basis points** (`mu_bps`).

Why

- Deterministic math, no float drift, easy comparisons/sums.
- Faster and simpler than Decimal for our scale.

Tradeoffs

- You must define rounding (half-up) and use it everywhere.
 - You’ll occasionally display values that the game would round differently—acceptable.
-

10.5 Run semantics: user-driven run boundaries + chat offset per run

Decision

- **User starts/stops** runs (primary workflow).
- On RunStart, store **chat_start_offset + chat_file_id** in `runs` .
- Parser reads only from that offset while run is ACTIVE.
- If no ACTIVE run → jump to EOF.

Why

- Users often do “random actions” outside tracking.
- Prevents “old garbage” being attributed to a run.
- Simple recovery: if app restarts and an ACTIVE run exists, resume exactly.

Tradeoffs

- If user forgets to start → those events are ignored (by design).
 - If chat log rotates, file_id logic must be robust.
-

10.6 Event correlation: deterministic, safe-first

Decision

- Claims from chat start as **PENDING_SCAN**.
- Approximate anchoring via **ProbeDrop queue matching** (time window).
- Deed scan is authoritative for coords/expiry.

Why

- Bulk dropping is common.
- We prefer “approximate until confirmed” over wrong precision.

Tradeoffs

- Before deed scan, claims may show as fuzzy circles only.
 - Correlation can fail; UI must handle unanchored claims.
-

10.7 Finder modes representation: bitmask, not “MIXED”

Decision

- Use `modes_bitmask` for probe drops and (optionally) run “modes seen”.
 - Example bits: ORE=1, ENMATTER=2, TREASURE=4

Why

- `MIXED` is semantically weak and causes cost ambiguity.
- Bitmask allows exact scaling rules and filtering.

Tradeoffs

- Slightly less human-readable in DB/API (UI converts to labels).
 - Requires a tiny shared constant definition.
-

10.8 OCR strategy: “cheap tracking” first, OCR when needed

Decision

- Prefer **cheap image features** (icon on/off, animation impulse) for high-frequency state.
- Use OCR for:
 - probes/ammo count,
 - status text,
 - deed parsing,
 - compass coords.

Why

- Tesseract is slow and can stall.
- Many UI states can be detected without OCR.

Tradeoffs

- Cheap tracking needs careful calibration and thresholds.
 - Some edge cases (amp unusable disables icons) must be handled explicitly.
-

10.9 Deed scanning: optional / toggleable

Decision

- Deed scanner can be **enabled/disabled** by user (and may be off by default).

Why

- Users might not want continuous OCR load.
- Many workflows scan deeds later or not at all.

Tradeoffs

- Without deed scans: no authoritative expiry/coords, only approximate claim UX.
 - UI must clearly show “PENDING_SCAN / incomplete”.
-

10.10 Resource classification: YAML file (user-editable), not an API

Decision

- Resource type mapping (ore/enmatter/treasure/undefined) stored in a **local YAML** edited by user.
- UI does not provide editing endpoints for this.

Why

- Simplest UX: notepad edit.
- Avoids building a config editor and validation flows early.

Tradeoffs

- Some users will mess up YAML → we need validation + error message.
 - Unknown resources default to `UNDEFINED` until user classifies.
-

10.11 Overlay windows UX: always-on-top, not click-through minimap

Decision

- Provide at least two overlay windows:
 - **Minimap/map overlay**: always-on-top, **not click-through**
 - **Cost/return overlay**: always-on-top, **can be click-through**
- No global hotkeys in v2.0 MVP.

Why

- Click-through minimap becomes painful to interact with.
- Cost overlay is mostly read-only numbers.

Tradeoffs

- No hotkeys means more manual window management.
 - Always-on-top can misbehave with fullscreen/exclusive modes (we handle best-effort).
-

10.12 Maps: offline tiles + planet coordinate transforms (in scope)

Decision

- Full offline map tiles are **in-scope**.
- Store planet grid offsets/transforms locally (per planet).

Why

- User already has tiles and planet coordinate grids.
- Big value for usability, no dependency on web services.

Tradeoffs

- Requires consistent coordinate transform validation.
 - Tile storage can be large; needs config for storage path.
-

10.13 Observability: keep debug data out of DB

Decision

- No raw chat lines, no raw OCR text in analytical tables.
- Debug mode may dump frames/logs to disk temporarily.

Why

- Privacy, and avoids bloating DB.
- Debug artifacts should be disposable.

Tradeoffs

- Harder to investigate rare parsing bugs without debug mode enabled.
 - Need good “diagnostics summary” in `/meta`.
-

10.14 Deferred (v2.0)

- Cloud sync and online backend
- Automatic HUD element detection every frame
- Loot MU pricing feeds
- Full claim-level loot attribution (best-effort later)
- Hotkeys / advanced overlay controls

11. Future – Cloud backend (Spring) & sync

11. Future – Cloud backend (Spring) & sync

This page describes a **non-blocking** future extension where the local MiningLog client can optionally synchronize selected, anonymized data to a cloud backend (Spring) for cross-device backup and aggregated statistics.

Important: This is **not** part of v2.0 MVP. The local client remains fully functional offline.

11.1 Goals

If enabled by the user, cloud sync should provide:

- **Backup & restore** of mining history across machines.
- **Optional aggregation** (global stats / planet resource distribution / tool usage patterns).
- A path to community features later (leaderboards etc.), without locking the local tool behind online services.

11.2 Non-goals

- No real-time multiplayer.
- No storing raw screenshots, OCR text dumps, or full chat logs.
- No mandatory accounts for local usage.
- No MU pricing feeds / “accounting accuracy” guarantees.

11.3 Privacy & Data Minimization

Data we may upload (candidate)

- Runs (time window, planet, modes seen, totals)
- Probe drops (time + approx location + radius + modes) **optional**
- Claims (resource, size, depth, coords if deed-scanned)
- Loot events (resource, quantity, TT value)
- Tool profile metadata (only if user chooses to sync config)

Data we must never upload

- Raw chat lines
- Raw OCR text
- Screenshots / image crops
- Player identifiers / nicknames
- Private chat logs

Anonymization approach

- Client is represented by a **random client_id**.
- If account/login exists, it's for sync access only, not EU identity.
- Optional “privacy levels”:
 - **Level 0**: totals only (no coords)
 - **Level 1**: claims + coarse coords (rounded)
 - **Level 2**: full deed coords (still no chat/screen)

11.4 Cloud Architecture (Proposed)

Components

- **Spring Boot API** (REST) + background ingestion jobs
- **PostgreSQL** as primary storage
- Optional: **S3-compatible object storage** (only for exported reports or debug bundles the user explicitly uploads)

Suggested modules

- `auth-service` (optional): user accounts, tokens
- `sync-service`: ingestion endpoints, conflict resolution, versioning
- `stats-service`: aggregation queries/materialized views

Keep it modular, but don't microservice it too early. One Spring app is enough initially.

11.5 Sync Model (Offline-first)

Sync is pull+push with a local cursor

- Local DB is source of truth.
- Cloud is a replica used for backup + aggregation.
- Sync runs:
 - manually (button), or
 - periodically (e.g. every N minutes) if user opts in.

Transport format

- Prefer **append-only event sync** rather than “send full tables”.
- We already have `event_outbox` locally → leverage it.

Key idea: sync the outbox (or a filtered subset) upstream.

11.6 Event-based Sync

Local requirement (future)

Maintain a second stream for cloud sync:

- `event_outbox` already exists for SSE.
- We can reuse it with:
 - a “syncable” flag, or
 - a `event_scope` enum: `LOCAL_UI | CLOUD_SYNC | BOTH`

Cloud ingestion

- Endpoint accepts a batch of events:
 - ordered by outbox `id`
 - idempotent via `dedup_key` or `(client_id, local_event_id)` uniqueness

Cloud stores:

- `ingested_events` table (for dedup + audit)
- derived entities (runs/claims/loot) in normalized tables, OR purely event-sourced with projections (later)

Recommendation: store entities + keep `ingested_events` only for dedup/audit. Event sourcing is overkill early.

11.7 Identity, Auth, and User Experience

MVP sync auth options

- **Option A (simplest):** user generates a “sync key” (API token) in UI; no passwords.
- **Option B:** classic email+password / OAuth later.

Client identity

- `client_id` generated on first run and persisted locally.
- Multiple devices can be linked to one account.

11.8 Conflict Handling

We avoid “true conflicts” by design:

- Runs are effectively time-bounded containers, but user can restart tracking.
- Ingestion is event-based and idempotent.

Conflicts scenarios:

- Two devices upload overlapping time ranges → allowed.
- Same event uploaded twice → dedup.
- Entity updates (claim becomes ACTIVE after deed scan) → model as an update event that supersedes previous state.

Rule: newest event wins per entity field if it has a later timestamp and valid schema version.

11.9 Data Dedup: claim_fingerprint / checksum

To support cross-client dedup and merging:

- Add `claim_fingerprint` (already planned) computed from stable deed fields when available:
 - `planet_code + lon + lat + depth + resource_code + size_idx + found_at_bucket`
- For “no deed scan” claims, fingerprint is unreliable → treat as local-only unless later enriched.

Cloud can enforce:

- unique index on `(account_id, claim_fingerprint)` to avoid duplicates in aggregated stats.

11.10 Schema Versioning and Compatibility

- All sync payloads include:
 - `schema_version`
 - `client_version`
- Cloud accepts:
 - current version + a small compatibility window (e.g. last 2 minor versions)
- Breaking change strategy:
 - new endpoint version `/sync/v2/...` or accept multiple schema versions and transform.

11.11 Rate limiting and Costs

- Cloud must protect itself:
 - per-account rate limiting
 - max batch sizes
 - max storage quotas (especially if coords are included)

Local side:

- upload in batches (e.g. 500–2000 events)
- exponential backoff on failures

11.12 Security Considerations (Pragmatic)

- TLS mandatory.
- Tokens stored encrypted on disk (OS keychain) if possible.
- Never accept arbitrary file uploads in early versions.
- No remote code execution “plugins” over sync. Ever.

11.13 Rollout Plan (Minimal Steps)

1. **Local outbox stability** (already needed for SSE)
2. Add `sync_cursor` locally: last uploaded outbox id per account
3. Cloud: implement `/sync/v1/events:batch`
4. Cloud: store `ingested_events` + basic entities
5. UI: “Enable sync” + “Sync now” + privacy level selection
6. Later: aggregated stats endpoints